

Data Bases II

Prof. Martinenghi Davide

Academic Year 2023-2024

Abstract

The course centers on Transactional Systems, exploring their importance and relevance. It introduces the concept of transactions and delves into the ACID properties that underpin them—atomicity, consistency, isolation, and durability.

Key concepts in concurrency control theory are examined, including schedules, serializability, and various notions of equivalence and complexity in testing, along with view-serializability and conflict-serializability. Advanced locking mechanisms such as two-phase locking, hierarchical locking, and deadlock analysis and resolution are discussed. The course also covers timestamp-based concurrency control, multi-version concurrency control, and locking implementations in commercial systems.

Database architecture topics include distributed databases, the concepts of fragmentation, allocation, and transparent access, query optimization, distributed transactions, and parallelism in database servers. The differences between shared-memory and shared-nothing approaches, as well as scale-up, speed-up, and performance benchmarking, are covered. Additionally, replicated databases are introduced, contrasting synchronous and asynchronous methods and symmetric versus primary-secondary replication strategies.

An exploration of database server internals covers buffer and page management, data organization with sequential, direct, and indexed structures, and data access techniques such as B and B+ trees and hashing functions. Access methods, including scans, ordering, and joins, are introduced, as well as fundamentals of query optimization, cost models, and execution plan selection using branch-and-bound methods. Topics in database administration and physical design, such as index and primary storage selection, are also addressed.

Active databases are introduced with the Event-Condition-Action paradigm, execution methods, trigger languages, and rule analysis, covering formal properties of active rule sets, termination, confluence, and observable determinism. Active rule design is discussed for purposes such as integrity maintenance, automated data derivation, and enforcing business rules.

Object-Relational Mapping concepts are presented, covering the mapping of entities, relationships, and hierarchies, the lifecycle of managed objects, and transactional support in business applications.

Finally, the course addresses ranking and its geometric interpretation in terms of data attributes, scoring functions, access methods, and algorithmic approaches to ranking. It discusses concepts such as dominance, skylines, k-skybands, and optimality, emphasizing both theoretical and practical perspectives in data management and middleware scenarios.

Contents

1	Concurrency	1
1.1	Introduction	1
1.2	Transactions	2
1.3	Concurrency theory	3
1.4	View serializability	4
1.5	Conflict serializability	5
1.6	Locks	7
1.6.1	Two-phase locking	7
1.6.2	Strict two-phase locking	8
1.7	Timestamps	8
1.7.1	Multi-version timestamp	10
1.8	Deadlocks	11
1.9	Summary	15
1.9.1	Isolation levels in SQL '99	16
1.9.2	Concurrency classes sets	16
2	Ranking	17
2.1	Introduction	17
2.2	History	17
2.3	Opaque rankings	18
2.4	Ranking queries	19
2.4.1	B-zero algorithm	25
2.4.2	A-zero algorithm	26
2.4.3	Threshold algorithm	27
2.4.4	No Random Access algorithm	28
2.4.5	Summary	30
2.5	Skyline queries	31
2.5.1	Block Nested Loop algorithm	31
2.5.2	Sort Filter Skyline algorithm	32
2.5.3	Summary	32
2.6	Comparison between ranking and skyline	32
3	Java Enterprise Edition	33
3.1	Introduction	33
3.2	Client-server architecture	33
3.3	Three-tier architecture	34
3.4	Java Enterprise Edition	34

3.5	Java Persistence API	35
3.5.1	Mapping	38
3.5.2	Relationship fetch mode	41
3.5.3	Cascading operations	42
3.6	JPA Entity Manager	42
3.6.1	Operations	43
3.6.2	Application architecture	44
3.7	Summary	45
4	Triggers	47
4.1	Definition and history	47
4.2	Introduction	47
4.3	Triggers definition	48
4.4	Triggers application	51
5	Physical databases	52
5.1	Introduction	52
5.2	Physical access structures	53
5.2.1	Sequential structures	55
5.2.2	Hash-based structures	56
5.2.3	Indexes	57
5.2.4	Tree-based structures	60
5.3	Query optimization	61
5.3.1	Relation profiles	62
5.3.2	Internal representation of queries	62
5.3.3	Cost-based Optimization	65
5.3.4	Approaches to Query Evaluation	65
6	Multidimensional indexing	66
6.1	Introduction	66
6.2	Data partitioning	67
6.2.1	Grid files	68
6.2.2	R-tree	69
6.3	Metric distances	76
6.3.1	M-tree	78

CHAPTER 1

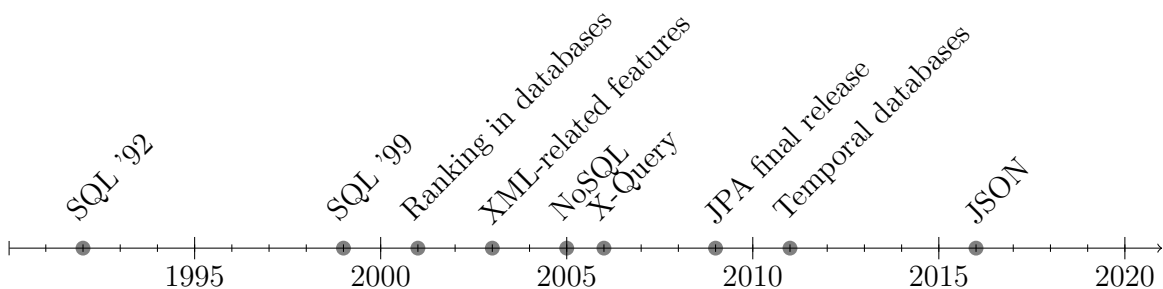
Concurrency

1.1 Introduction

Definition (*Data Base Management System*). A Database Management System (DBMS) is a software product designed to manage collections of data that possess the following characteristics:

- *Large*: the data collections are significantly larger than the central memory available on the computers that run the software.
- *Persistent*: the data has a lifespan independent of individual executions of the programs that access it.
- *Shared*: the data is accessed by multiple applications simultaneously.
- *Reliable*: the system ensures tolerance to hardware and software failures.
- *Respectful of data ownership*: access is disciplined and controlled.

Key developments in DBMS technology are outlined in the following chronological timeline:



The architecture of a DBMS is illustrated in the figure below:

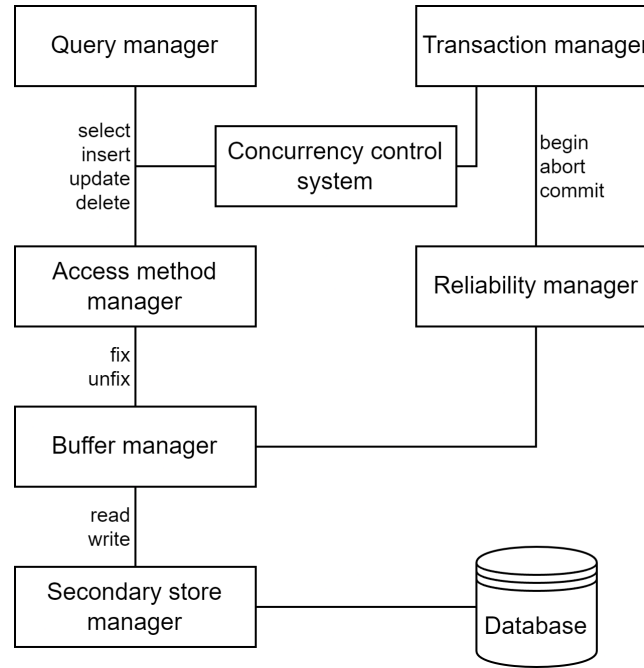


Figure 1.1: DBMS architecture

1.2 Transactions

Definition (*Transaction*). A transaction is an elementary, atomic unit of work performed by an application.

Each transaction is conceptually encapsulated within two commands: **begin** and **end**. In the context of a transaction, the conclusion of the process is indicated by executing one of the following commands **commit** or **rollback**.

Definition (*On-Line Transaction Processing*). On-Line Transaction Processing (OLTP) is a system that supports the execution of transactions on behalf of concurrent applications.

The application has the capability to execute multiple transactions, with transactions being an integral part of the application.

ACID properties These transactions adhere to the ACID properties:

1. *Atomicity*: a transaction represents an indivisible unit of execution, ensuring that either all operations within the transaction are executed or none at all. The commitment of the transaction, marked by the execution of the commit command, signifies the successful conclusion. Any error occurring before the commit should trigger a rollback, and errors occurring afterward should not impact the transaction. The rollback can be initiated by either a rollback statement or the DBMS. In the event of a rollback, the executed work must be undone, restoring the database to its state before the transaction's initiation. It becomes the responsibility of the application to determine whether an aborted transaction needs to be redone.
2. *Consistency*: transactions must adhere to the integrity constraints of the database. If the initial state S_0 is consistent, the final state S_f must also be consistent. This consistency requirement, however, does not necessarily extend to intermediate states S_i .

3. *Isolation*: the execution of a transaction must remain independent of concurrently executing transactions. This ensures that the outcome of one transaction does not affect the execution of others.
4. *Durability*: the effects of a successfully committed transaction are permanent, enduring indefinitely and remaining unaffected by any system faults.

Each of these properties is assured by corresponding components of the displayed DBMS. Specifically, we have that:

Property	Actions	Architectural element
Atomicity	Abort-rollback-restart	Query manager
Consistency	Integrity checking	Integrity control system
Isolation	Concurrency control	Concurrency control system
Durability	Recovery management	Reliability manager

A DBMS typically manage multiple applications. An essential metric for evaluating the workload of a DBMS is the number of transactions per second that it can effectively handle. Efficient database utilization requires the DBMS to manage concurrency effectively while preventing the occurrence of anomalies. This is achieved through a concurrency control system, which is responsible for determining the order in which various transactions are scheduled.

1.3 Concurrency theory

The anomalies resulting from incorrect scheduling include:

- *Lost updates*: update applied from a state that ignores a preceding update, resulting in the loss of the earlier update.
- *Dirty reads*: an uncommitted value is used to update the data.
- *Non-repeatable reads*: another transaction updates a previously read value.
- *Phantom updates*: another transaction updates data that contributes to a previously valid constraint.
- *Phantom inserts*: another transaction inserts data that contributes to a previously read datum.

Definition (Model). A model is an abstraction of a system, object or process, which purposely disregards details to simplify the investigation of relevant properties.

Concurrency theory is founded on a model of transaction and concurrency control principles that aids in comprehending real systems. Actual systems leverage implementation-level mechanisms to attain desirable properties postulated by the theory.

Definition (Operation). An operation consist in a reading or in a writing of a specific datum by a specific transaction.

Definition (Schedule). A schedule is a sequence of operations performed by concurrent transactions that respects the order of operations of each transaction.

Schedule Transactions can be categorized as serial, interleaved, or nested. Let N_S and N_D be respectively the number of serial schedules and distinct schedules for n transactions $\langle T_1, \dots, T_n \rangle$ each with k_i operations. Then:

$$N_S = n! \quad N_D = \frac{(\sum_{i=1}^n k_i)!}{\prod_{i=1}^n (k_i!)}$$

Example:

For two transactions, T_1 and T_2 , there are six possible schedules, with only two being serial:

1. $r_1(x)w_1(x)r_2(z)w_2(z) \rightarrow$ serial
2. $r_2(z)w_2(z)r_1(x)w_1(x) \rightarrow$ serial
3. $r_1(x)r_2(z)w_1(x)w_2(z) \rightarrow$ nested
4. $r_2(z)r_1(x)w_2(z)w_1(x) \rightarrow$ nested
5. $r_1(x)r_2(z)w_2(z)w_1(x) \rightarrow$ interleaved
6. $r_2(z)r_1(x)w_1(x)w_2(z) \rightarrow$ interleaved

Concurrency control must reject all schedules that lead to anomalies.

Definition (*Scheduler*). The scheduler is a component that accepts or rejects operations requested by the transactions.

Definition (*Serial schedule*). The serial schedule is a schedule in which the actions of each transaction occur in a contiguous sequence.

A serializable schedule leaves the database in the same state as some serial schedule of the same transactions, making it correct. To introduce other classes, two initial assumptions are made:

- The transactions are observed a posteriori.
- The transactions considered are only the committed ones.

1.4 View serializability

Definition (*Read from*). The operation $r_i(x)$ reads-from $w_j(x)$ when $w_j(x)$ precedes $r_i(x)$ and there is no $w_k(x)$ in the schedule between $r_i(x)$ and $w_j(x)$.

Definition (*Final write*). The operation $w_i(x)$ in a schedule is a final write if it is the last write on x that occurs in it.

Definition (*View equivalence*). Two schedules are said to be view equivalent if they have the same operations, reads-from relationships and final writes.

Definition (*View Serializable*). A schedule is View Serializable (VSR) if it is view-equivalent to a serial schedule of the same transactions.

The value written by $w_j(x)$ could be uncommitted when $r_i(x)$ reads it, but we are certain that it will be committed (commit-projection hypothesis).

Example:

Consider the following schedules:

- $S_1 : w_0(x)r_2(x)r_1(x)w_2(x)w_2(z)$
- $S_2 : w_0(x)r_1(x)r_2(x)w_2(x)w_2(z)$
- $S_3 : w_0(x)r_1(x)w_1(x)r_2(x)w_1(z)$
- $S_4 : w_0(x)r_1(x)w_1(x)w_1(z)r_2(x)$
- $S_5 : r_1(x)r_2(x)w_1(x)w_2(x)$
- $S_6 : r_1(x)r_2(x)w_2(x)r_1(x)$
- $S_7 : r_1(x)r_1(y)r_2(z)r_2(y)w_2(y)w_2(z)r_1(z)$

Only S_2 and S_3 are serial. S_1 is view-equivalent to the serial schedule S_2 (thus, VSR). S_3 is not view-equivalent to S_2 (due to different operations) but is view-equivalent to the serial schedule S_4 , making it VSR.

S_5 corresponds to a lost update, S_6 corresponds to a non-repeatable read, and S_7 corresponds to a phantom update. All these schedules are non VSR.

Additionally, consider the following schedules:

- $S_a : w_0(x)r_1(x)w_0(z)r_1(z)r_2(x)w_0(y)r_3(z)w_3(z)w_2(y)w_1(x)w_3(y)$
- $S_b : w_0(x)w_0(z)w_0(y)r_2(x)w_2(y)r_1(x)r_1(z)w_1(x)r_3(z)w_3(z)w_3(y)$
- $S_c : w_0(x)w_0(z)w_0(y)r_2(x)w_2(y)r_3(z)w_3(z)w_3(y)r_1(x)r_1(z)w_1(x)$

S_a and S_b are view-equivalent because all the reads-from relationships and final writes are the same. Specifically, we have:

- Reads-from: $r_1(x)$ from $w_0(x)$, $r_1(z)$ from $w_0(z)$, $r_2(x)$ from $w_0(x)$, $r_3(z)$ from $w_0(z)$.
- Final writes: $w_1(x)$, $w_3(y)$, $w_3(z)$.

However, S_a and S_c are not view-equivalent because not all the reads-from relationships are the same.

Deciding if a generic schedule is in VSR is an $\mathcal{NP}$ -complete problem. Therefore, a more stringent definition is needed, which is easier to check. This new definition may result in rejecting some schedules that would be acceptable under view-serializability but not under the stricter criterion.

1.5 Conflict serializability

Definition (Conflict). Two operations o_i and o_j ($i \neq j$) are in conflict if they address the same resource and at least one of them is write. There are two possible cases: read-write conflicts or write-write conflicts.

Definition (Conflict equivalent). Two schedules are conflict equivalent ($S_i \approx_C S_j$) if S_i and S_j contain the same operations and in all the conflicting pairs the transactions occur in the same

order.

Definition (*Conflict Serializable*). A schedule is Conflict Serializable (CSR) if and only if it is conflict equivalent to a serial schedule of the same transactions.

Property 1.5.1. CSR is a strict subset of VSR.

Proof. Consider the schedule $S = r_1(x)w_2(x)w_1(x)w_3(x)$, which is VSR but not CSR. It can be verified that there is no CSR serial schedule. $\square$

Property 1.5.2. CSR implies VSR.

Proof. Assuming $S_1 \approx_C S_2$, we can prove that $S_1 \approx_V S_2$. S_1 and S_2 must have: the same final writes (if not, there would be at least two writes in a different order, and since two writes are conflicting operations, the schedules would not be CSR), and the same reads-from relations (if not, there would be at least one pair of conflicting operations in a different order, and therefore $\approx_C$ would be violated). $\square$

Conflict graph To assess view-serializability, a conflict graph is constructed, with one node for each transaction T_i , and an arc from T_i to T_j if there exists at least one conflict between an operation o_i of T_i and an operation o_j of T_j such that o_i precedes o_j .

Theorem 1.5.1. A schedule is conflict-serializable if and only if its conflict graph is acyclic.

Example:

Consider the schedule

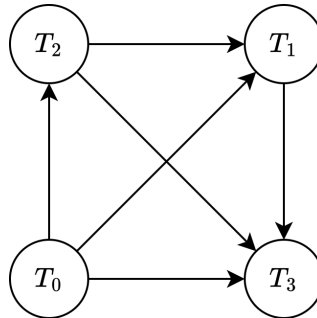
$$S : w_0(x)r_1(x)w_0(z)r_1(z)r_2(x)w_0(y)r_3(z)w_3(z)w_2(y)w_1(x)w_3(y)$$

To test conflict serializability, follow these steps:

1. Create nodes based on the number of transactions in the schedule.
2. Group operations based on the requested resource.
3. Check all write-write and read-write relationships in each subset and add arcs accordingly.

For the given example, we obtain:

- $x : w_0r_1r_2w_1$
- $y : w_0w_2w_3$
- $z : w_0r_1r_3w_3$



Since there are no cycles in the graph, the schedule is conflict-serializable.

Proof. We prove that CSR implies acyclicity. Consider a schedule S in CSR. As such, it is $\approx_C$ to a serial schedule. Without loss of generality we can label the transactions of S to say that their order in the serial schedule is: $T_1T_2 \dots T_n$. Since the serial schedule has all conflicting pairs in the same order as schedule S , in the conflict graph there can only be arcs (i, j) , with $i < j$. Then the graph is acyclic, as a cycle requires at least an arc (i, j) with $i > j$. $\square$

Proof. We prove that acyclicity implies CSR. If S 's graph is acyclic then it induces a topological (partial) ordering on its nodes. The same partial order exists on the transactions of S . Any serial schedule whose transactions are ordered according to the partial order is CSR to S , because for all conflicting pairs (i, j) it is always $i < j$. $\square$

1.6 Locks

The prevalent method in commercial systems is the use of locking. A transaction is considered well-formed concerning locking if:

- Read operations are preceded by `r_lock` (shared) and followed by `unlock`.
- Write operations are preceded by `w_lock` (exclusive) and followed by `unlock`.

In both cases, unlocking can be delayed with respect to the completion of the operations. Consequently, every object can be in one of three states: `free`, `r_locked`, or `w_locked`. Transactions that first read and then write an object may acquire a `w_lock` already when reading or acquire a `r_lock` first and then upgrade it into a `w_lock` (escalation).

Conflict table The lock manager receives requests from transactions and allocates resources based on the conflict table:

Request	Resource status		
	<i>FREE</i>	<i>R_LOCKED</i>	<i>W_LOCKED</i>
<i>r_lock</i>	✓ R_LOCKED	✓ R_LOCKED($n++$)	✗ W_LOCKED
<i>w_lock</i>	✓ W_LOCKED	✗ R_LOCKED	✗ W_LOCKED
<i>unlock</i>	ERROR	✓ $n--$	✓ FREE

Lock tables The locking system is implemented via lock tables, which are hash tables indexing lockable items via hashing. Each locked item has an associated linked list, with each node representing the transaction that requested the lock, the lock mode, and the current status. Each new lock request for the data item is appended to the list.

1.6.1 Two-phase locking

The previously presented locking mechanism does not eliminate anomalies caused by non-repeatable reads. To address this issue, a two-phase rule can be employed, requiring that a transaction cannot acquire any other lock after releasing one. This approach involves three phases: acquiring all locks, executing operations, and finally, unlocking.

Definition (*Two-phase locking*). The class of two-phase locking is the set of all schedules generated by a scheduler that: only processes well-formed transactions, grant locks according to the conflict table, and checks that all transactions apply the two-phase rule.

Property 1.6.1. 2PL is a strict subset of CSR.

Proof. We assume that a schedule S is 2PL. Consider, for each transaction, the moment in which it holds all locks and is going to release the first one. We sort the transactions by this temporal value and consider the corresponding serial schedule S' . We want to prove by contradiction that S is C to S' :

$$S' \approx_C S, \dots$$

Consider a generic conflict $o_i \rightarrow o_j$ in S' with $o_i \in T_i$, $o_j \in T_j$, and $i < j$. By definition of conflict, o_i and o_j address the same resource r , and at least one of them is write. The two operations cannot occur in reverse order of S . This proves that all 2PL schedules are VSR. $\square$

Property 1.6.2. 2PL implies CSR.

In this state, the remaining anomalies are limited to phantom inserts (requiring predicate locks) and dirty reads.

1.6.2 Strict two-phase locking

Definition (*Strict two-phase locking*). In strict two-phase locking (or long duration locks) we also have that locks held by a transaction can be released only after commit or rollback.

This locking variant is employed in many commercial DBMS when a high level of isolation is necessary.

Predicate locking To counteract phantom inserts, a lock should also be applied to future data using predicate locks. When a predicate lock is placed on a resource, other transactions are restricted from inserting, deleting, or updating any tuple that satisfies this predicate.

1.7 Timestamps

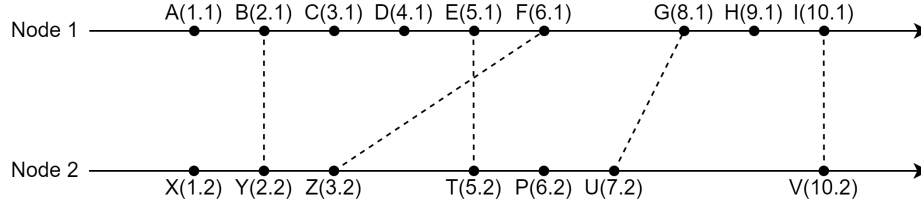
Locking assumes collisions will occur, although in reality, collisions are infrequent. Optimistic concurrency control methods like timestamps can be employed to address this. Timestamps are identifiers that establish a total ordering of a system's events. Each transaction is assigned a timestamp representing its initiation time, enabling transactions to be ordered based on their timestamps. A schedule is accepted only if it aligns with the serial ordering of transactions induced by their timestamps. Timestamps, given by a system's function upon request, have the syntax:

$$\langle \text{event-id.node-id} \rangle$$

The synchronization algorithm, known as the Lamport method, relies on the send-receive of messages. It ensures that a message from the future cannot be received. If this occurs, the bumping rule is employed to adjust the timestamp of the receiving event beyond that of the sending event.

Example:

Timestamp assignment at two different nodes might look like the following.



Counters The scheduler uses two counters: one for writes ($WTM(x)$) and another for reads ($RTM(x)$). Read and write requests are tagged with the timestamp of the requesting transaction.

A read is accepted if its timestamp is higher than the last write timestamp. In this case, $RTM(x) = \max(RTM(x), ts)$.

A write is accepted if its timestamp is higher than the last write and last read timestamps. In this case $WTM(x) = ts$.

In case a read or a write is not accepted, the transaction is killed. However, these rules may lead to excessive transaction killings.

Example:

Assuming $RTM(x) = 7$ and $WTM(x) = 4$, consider the following schedule:

$$S = r_6(x)r_8(x)r_9(x)w_8(x)w_{11}(x)r_{10}(x)$$

Using timestamps, we obtain:

Request	Response	New value
$r_6(x)$	✓	-
$r_8(x)$	✓	$RTM(x) = 8$
$r_9(x)$	✓	$RTM(x) = 9$
$w_8(x)$	✗	T_8 killed
$w_{11}(x)$	✓	$WTM(x) = 11$
$r_{10}(x)$	✗	T_{10} killed

Comparing 2PL to is challenging, and there is no subset relationship between the two. However, TS implies CSR.

Proof. Let S be a TS schedule of T_1 and T_2 . Suppose S is not CSR, which implies that it contains a cycle between T_1 and T_2 . S contains $op_1(x)$, $op_2(x)$ where at least one is a write. S contains also $op_2(y)$, $op_1(y)$ where at least one is a write. When $op_1(y)$ arrives we have two possibilities. If $op_1(y)$ is a read, T_1 is killed by TS because it tries to read a value written by a younger transaction, so it is a contradiction. If $op_1(y)$ is a write, T_1 is killed no matter what $op_2(y)$ is, because it tries to write a value already read or written by a younger transaction, so it is a contradiction. $\square$

Transaction abort Basic TS-based control considers only committed transactions, ignoring aborted transactions. If aborts occur, dirty reads may happen. To handle dirty reads, a variant of basic TS must be used. A transaction T_i issuing $r_{ts}(x)$ or $w_{ts}(x)$ such that $ts >$

$WTM(x)$ delays its read or write operation until the transaction T' that wrote the value of x has committed or aborted. This is similar to long-duration write locks.

Action	2PL	TS
Transaction	Wait	Killed and restarted
Serialization	Imposed by conflicts	Imposed by timestamp
Delay	Long (strict version)	Long
Deadlocks	Possible	Prevented

Since restarting a transaction is costlier than waiting, 2PL is preferable when used alone. Commercial systems often combine these techniques to leverage the best features of both.

Thomas rule To reduce the number of killings, the Thomas rule can be used, altering the rule for write operations.

A write is accepted if its timestamp is higher than the last read timestamp, and the write timestamp is updated. A write is skipped if its timestamp is lower than the last write timestamp.

1.7.1 Multi-version timestamp

The concept of multi-versioning involves generating new versions with each write operation, and reads access the relevant version. Each write produces new copies, each with a new write timestamp ($WTM(x)$), ensuring that each object x always has $N \geq 1$ active versions. A unique global read timestamp ($RTM(x)$) is maintained, and old versions are discarded when there are no transactions requiring their values.

The reads are always accepted and a copy x_k is selected for reading, where:

- If $ts \geq WTM_N(x)$, then $k = N$.
- Otherwise, k is chosen such that $WTM_k(x) \leq ts < WTM_{k+1}(x)$.

The write is accepted if its timestamp is higher than the last read timestamp, and the write timestamp is updated.

Example:

Assuming $RTM(x) = 7$, $N = 1$ and $WTM_1(x) = 4$, consider the schedule:

$$S = r_6(x)r_8(x)r_9(x)w_8(x)w_{11}(x)r_{10}(x)r_{12}(x)w_{14}(x)w_{13}(x)$$

Using multi-versioning, the results are:

Request	Response	New value
$r_6(x)$	✓	-
$r_8(x)$	✓	$RTM(x) = 8$
$r_9(x)$	✓	$RTM(x) = 9$
$w_8(x)$	×	T_8 killed
$w_{11}(x)$	✓	$WTM_2(x) = 11$, $N = 2$
$r_{10}(x)$	✓ on $x_{(1)}$	$RTM(x) = 10$
$r_{12}(x)$	✓ on $x_{(2)}$	$RTM(x) = 12$
$w_{14}(x)$	✓	$WTM_3(x) = 14$, $N = 3$
$w_{13}(x)$	✓	$WTM_4(x) = 14$, $N = 4$

Multi-versioning in practice In practice, the rule set is modified slightly. The reads are always accepted and a copy x_k is selected for reading, where:

- If $ts \geq WTM_N(x)$, then $k = N$.
- Otherwise, k is chosen such that $WTM_k(x) \leq ts < WTM_{k+1}(x)$.

The write is accepted if its timestamp is higher than the last read and last write timestamps, and the write timestamp is updated.

Example:

Assuming $RTM(x) = 7$, $N = 1$ and $WTM_1(x) = 4$, consider the schedule:

$$S = r_6(x)r_8(x)r_9(x)w_8(x)w_{11}(x)r_{10}(x)r_{12}(x)w_{14}(x)w_{13}(x)$$

Using multi-versioning, the results are:

Request	Response	New value
$r_6(x)$	✓	-
$r_8(x)$	✓	$RTM(x) = 8$
$r_9(x)$	✓	$RTM(x) = 9$
$w_8(x)$	✗	T_8 killed
$w_{11}(x)$	✓	$WTM_2(x) = 11$, $N = 2$
$r_{10}(x)$	✓ on $x_{(1)}$	$RTM(x) = 10$
$r_{12}(x)$	✓ on $x_{(2)}$	$RTM(x) = 12$
$w_{14}(x)$	✓	$WTM_3(x) = 14$, $N = 3$
$w_{13}(x)$	✗	T_{13} killed

Isolation levels The implementation of TS-multi opens the door to introducing another isolation level in the DBMS (DBMS), known as snapshot isolation. In this level, only Write Timestamp ($WTM(x)$) is utilized. The rule applied in snapshot isolation dictates that every transaction reads the version consistent with its timestamp and defers writes until the end. If the scheduler detects conflicts between the writes of a transaction and the writes of other concurrent transactions after the snapshot timestamp, it aborts. It's essential to note that while snapshot isolation provides certain guarantees, it does not ensure serializability, and a new anomaly known as write skew (non-determinism) can occur.

1.8 Deadlocks

A deadlock arises when concurrent transactions hold and request resources held by other transactions.

Definition (*Lock graph*). A lock graph is a bipartite graph in which nodes are resources or transactions and arcs are lock requests or lock assignments.

Definition (*Wait-for graph*). A wait-for graph is a graph in which nodes are transactions and arcs are waits for relationships.

A deadlock is indicated by a cycle in the wait-for graph of transactions.

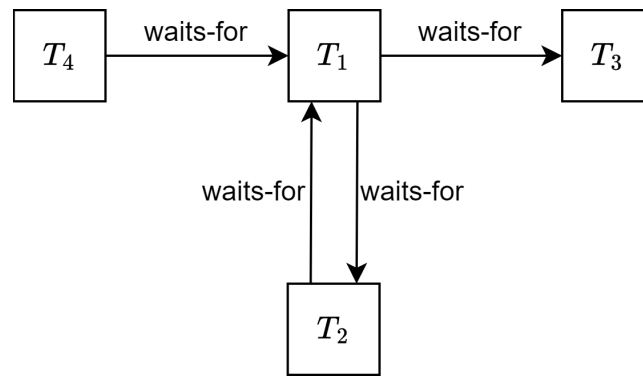


Figure 1.2: An example of deadlock in the wait-for graph

It is possible to solve deadlocks in three different ways:

- *Timeout*: a transaction is terminated and restarted after a specified waiting time, determined by the system manager.
- *Deadlock prevention*: kills transactions that could cause cycles. It is implemented in two ways:
 1. *Resource-based prevention* puts restrictions on lock requests. The idea is that every transaction requests all resources at once, and only once. The main problem is that it's not easy for transactions to anticipate all requests.
 2. *Transaction-based prevention* puts restrictions on transactions' IDs. Assigning IDs to transactions incrementally allows to give an age to each one. It is possible to choose to kill the holding transaction (preemptive) or the requesting one (non-preemptive). The main problem is that the number of killings is too big.
- *Deadlock detection*: it can be implemented with various algorithms and used for distributed resources.

Dependency graph The distributed dependency graph is a wait-for graph where external call nodes represent a sub-transaction activating another sub-transaction at a different node. The arrow shows a wait-for relation among local transactions. If one term is an external call, either the source is waited for by a remote transaction or waits for a remote transaction.

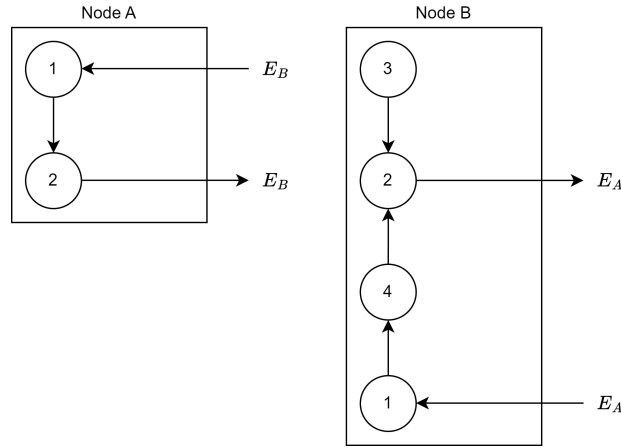
Obermarck's algorithm The Obermarck's algorithm needs the following assumptions:

- Transactions execute on a single main node.
- Transactions may be decomposed in sub-transactions running on other nodes.
- When a transaction spawns a sub-transaction it suspends work until the latter completes.
- Two wait-for relationships:
 - T_i waits for T_j on the same node because T_i needs a datum locked by T_j .
 - A sub-transaction of T_i waits for another sub-transaction of T_i running on a different node.

The goal of this algorithm is to detect a potential deadlock looking only at the local view of a node. Nodes exchange information and update their local graph based on the received information. Node A sends its local info to a node B only if it contains a transaction T_i that is waited for from another remote transaction and waits for a transaction T_j active on B and $i > j$.

Example:

Consider the given distributed dependency graph:



In this graph, a potential deadlock is indicated by cycles. Specifically, we observe that T_{2a} waits for T_{1a} (data lock), which, in turn, waits for T_{1b} (call), leading to a sequence where T_{2b} (data locks) waits for T_{2a} (call).

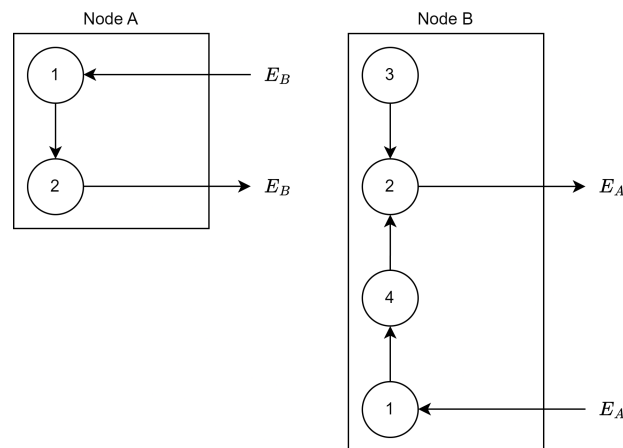
In this case the node A dispatches information to B , in fact we have $E_b \rightarrow T_2 \rightarrow T_1 \rightarrow E_b$ and node B cannot dispatch information to A , because the forwarding rule is not respected: $E_a \rightarrow T_1 \rightarrow T_2 \rightarrow E_a$.

The Obermarck's algorithm runs periodically at each node and consists in four steps:

1. Obtain graph info from the previous nodes.
2. Update the local graph by merging the received information.
3. Check for cycles among transactions, indicating potential deadlocks. If found, select one transaction in the cycle and terminate it.
4. Send the updated graph info to the next nodes.

Example:

Given the following distributed system:



Let's apply the Obermarck's algorithm:

1. Use the forwarding rule, in this case we have:
 - At Node A: $E_b \rightarrow T_2 \rightarrow T_1 \rightarrow E_b$ info sent to Node B
 - At Node B: $E_a \rightarrow T_1 \rightarrow T_2 \rightarrow E_a$ info not sent ($i < j$).
2. At node B there is the updated info $E_b \rightarrow T_2 \rightarrow T_1 \rightarrow E_b$. and it is added to the wait-for graph.
3. At node B a deadlock is detected (cycle between T_1 and T_2) and T_1 , T_2 or T_4 are killed.
4. Updated information are sent to all nodes.

Four variants based on different conventions regarding message conditions and receivers.

	Message condition	Message receiver
A	$i > j$	Following node
B	$i > j$	Preceding node
C	$i < j$	Following node
D	$i < j$	Preceding node

Practical application In practical scenarios, the likelihood of encountering deadlocks (n^{-2}) is considerably lower than the probability of conflicts (n^{-1}). Various techniques are employed to minimize the occurrence of deadlocks:

- *Update lock*: the most frequent deadlock occurs when two concurrent transactions start by reading the same resources and then decide to write and try to upgrade their lock to write on the resource. To avoid this situation, systems offer the update lock, that is used by transactions that will read and then write. The lock table become:

Request	Resource status			
	FREE	SHARED	UPDATE	EXCLUSIVE
Shared lock	✓	✓	✓	×
Update lock	✓	✓	×	×
Exclusive lock	✓	×	×	×

- *Hierarchical lock*: locks can be specified with different granularity. The objective of this is to lock the minimum amount of data and recognize conflicts as soon as possible. The method used to do so consists in asking locks on hierarchical resources by requesting resources top-down until the right level is obtained and releasing locks bottom-up. This is done by using five locking modes: shared, exclusive, ISL (intention of locking a sub-element of the current element in shared mode), IXL (intention of locking a sub-element of the current element in exclusive mode), and SIXL (lock of the element in shared mode with intention of locking a sub-element in exclusive mode). The lock table is modified as follows:

Request	Resource status					
	<i>FREE</i>	<i>ISL</i>	<i>IXL</i>	<i>SL</i>	<i>SIXL</i>	<i>XL</i>
<i>ISL</i>	✓	✓	✓	✓	✓	×
<i>IXL</i>	✓	✓	✓	×	×	×
<i>SL</i>	✓	✓	×	✓	×	×
<i>SIXL</i>	✓	✓	×	×	×	×
<i>XL</i>	✓	×	×	×	×	×

Example:

Given a table *X* with eight tuples divided in two pages:

P1	P2
<i>t1</i>	<i>t5</i>
<i>t2</i>	<i>t6</i>
<i>t3</i>	<i>t7</i>
<i>t4</i>	<i>t8</i>

And two transactions with the following schedules:

$$T_1 = r(P1) \ w(t3) \ r(t8)$$

$$T_2 = r(t2) \ r(t4) \ w(t5) \ w(t6)$$

We can see that they are not in a read-write conflict (because they are independent of the order). Without hierarchical locking both transactions needs to operate on the same table, so the concurrency will be almost useless in this case. But with this technique, calling *X* the table, we have that the transaction acquires the following locks:

$$T_1 : \text{IXL}(\text{root}) \ \text{SIXL}(P1) \ \text{XL}(t3) \ \text{ISL}(P2) \ \text{SL}(t8)$$

$$T_2 : \text{IXL}(\text{root}) \ \text{ISL}(P1) \ \text{SL}(t2) \ \text{SL}(t4) \ \text{IXL}(P2) \ \text{XL}(t5) \ \text{XL}(t6)$$

1.9 Summary

Checking conflict-serializability would be efficient if the conflict graph were known from the outset, but in practice, it is typically not. Therefore, a scheduler must work online. It is impractical to maintain, update, and check the conflict graph at each operation request. Additionally, the assumption that concurrency control can rely solely on the commit-projection of

the schedule is unrealistic because aborts do occur. Thus, an online scheduler needs a simple decision criterion that avoids as many anomalies as possible with minimal overhead.

Online concurrency control In the realm of online concurrency control, considering arrival sequences is crucial. The concurrency control system translates an arrival sequence into an effective a posteriori schedule. Two main families of techniques are commonly employed for online scheduling:

- Pessimistic techniques (*locks*): if a resource is taken, make the requester wait or pre-empt the holder.
- Optimistic techniques (*timestamps*): serve as many requests as possible, possibly using out-of-date versions of the data.

In practice, commercial systems often combine elements from both pessimistic and optimistic approaches to leverage the strengths of each.

1.9.1 Isolation levels in SQL '99

SQL defines transaction isolation levels that specify the anomalies to be prevented when running at each level:

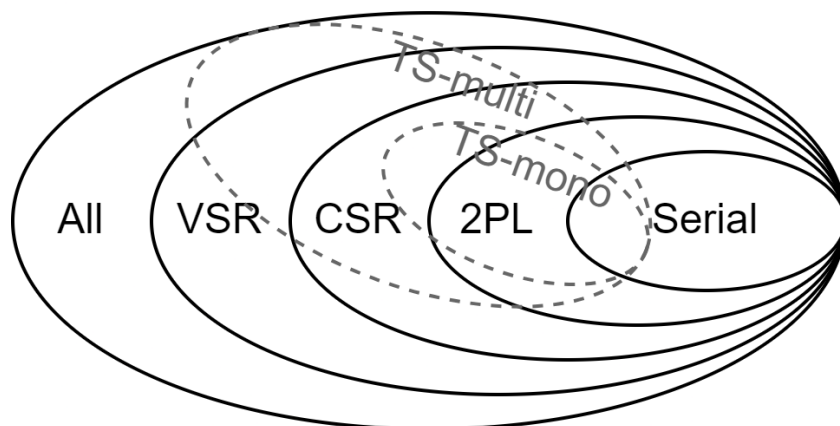
	Dirty read	Non-repeatable read	Phantoms
Read uncommitted	✓	✓	✓
Read committed	×	✓	✓
Repeatable reads	×	×	✓ (insert)
Serializable	×	×	×

The four levels are implemented respectively with: no read locks, normal read locks, strict read locks, and strict locks with predicate locks. **Serializable** is not the default because its strictness can lead to the following problems:

- Deadlock: two or more transactions are in endless mutual wait.
- Starvation: a single transaction is in endless wait.

1.9.2 Concurrency classes sets

Here is the final configuration for the sets of concurrency classes:



CHAPTER 2

Ranking

2.1 Introduction

Achieving optimal results from data can be accomplished through the application of ranking, involving the simultaneous optimization of various criteria. A typical formulation for a multi-objective problem is as follows: with N objects characterized by d attributes, the objective is to identify the top k objects.

Methods The primary methods for addressing multi-objective optimization include:

- *Ranking queries*: this approach involves selecting the top k objects based on a specified scoring function.
- *Skyline queries*: this method focuses on choosing the set of non-dominated objects, ensuring they are not inferior in any criterion.

2.2 History

In the 13th century, the problem of elections and polls was introduced. Borda proposed a solution involving assigning penalty points based on the position of candidates in a voter's ranking, aiming for the candidate with the lowest overall penalty to win. On the other hand, Condorcet suggested determining the winner as the candidate who defeats every other candidate in pairwise majority rule elections.

Example:

Given ten voters and three candidates with the following votes:

1	2	3	4	5	6	7	8	9	10
A	A	A	A	A	A	C	C	C	C
C	C	C	C	C	C	B	B	B	B
B	B	B	B	B	B	A	A	A	A

For Borda we have:

- $A : 1 \cdot 6 + 3 \cdot 4 = 18$

- $B : 3 \cdot 6 + 2 \cdot 4 = 26$
- $C : 2 \cdot 6 + 1 \cdot 4 = 16$ (winner)

However, for Condorcet, A wins in pairwise majority. The winner depends on the method used.

In 1950, Arrow proposed the axiomatic approach, defining aggregation as axioms. He formulated Arrow's paradox, stating that no rank-order electoral system can satisfy the following fairness criteria simultaneously:

- No dictatorship (nobody determines, alone, the group's preference).
- If all prefer X to Y , then the group prefers X to Y .
- If, for all voters, the preference between X and Y is unchanged, then the group preference between X and Y is unchanged.

To address this paradox, researchers introduced the metric approach. This involves finding a new ranking R minimizing the total distance to the initial rankings $R_1, \dots, R_n$. Distances between rankings can be measured using methods such as:

- Kendall tau distance $K(R_1, R_2)$, defined as the number of exchanges in a bubble sort to convert R_1 to R_2 .
- Spearman's foot-rule distance $F(R_1, R_2)$, which adds up the distance between the ranks of the same item in the two rankings.

Finding an exact solution is computationally hard for Kendall tau ($\mathcal{NP}$ -complete), but tractable for Spearman's foot-rule ($\mathcal{P}$ time). These distances are related as follows:

$$K(R_1, R_2) \leq F(R_1, R_2) \leq 2K(R_1, R_2)$$

Efficient approximations for $F(R_1, R_2)$ are also possible.

2.3 Opaque rankings

The concept of opaque rankings focuses only on positions without considering associated scores.

MedRank algorithm The MedRank algorithm, inspired by the median, offers an approximation of the foot-rule optimal aggregation. The algorithm takes an integer k and a ranked list $R_1, \dots, R_m$ of N elements as inputs, producing the top k elements based on the median ranking. The algorithm proceeds as follows:

1. Use sorted accesses in each list, extracting one element at a time until there are k elements occurring in more than $\frac{m}{2}$ lists.
2. Identify these as the top k elements.

Definition (*Depth*). The maximum number of sorted accesses made on each list is called the depth reached by the algorithm.

Example:

Consider sorting the following hotels based on three criteria using the MedRank algorithm. The hotels' reviews are as follows:

Price	Rating	Distance
Ibis	Crillon	Le Roch
Etap	Novotel	Lodge In
Novotel	Sheraton	Ritz
Mercure	Hilton	Lutetia
Hilton	Ibis	Novotel
Sheraton	Ritz	Sheraton
Crillon	Lutetia	Mercure
...	...	...

Using MedRank with $k = 3$, the resulting rank is:

Top k hotels	Median rank
Novotel	$\text{median}\{2, 3, 5\} = 3$
Hilton	$\text{median}\{4, 5, ?\} = 5$
Ibis	$\text{median}\{1, 5, ?\} = 5$

The depth in this case is equal to five.

Definition (*Optimal*). An algorithm is optimal if its execution cost is never worse than any other algorithm on any input.

Definition (*Instance optimal*). An algorithm is instance-optimal if is the best possible algorithm on every input of a specific instance.

Property 2.3.1. MedRank is instace-optimal on ordered lists.

Theorem 2.3.1 (Instance optimality). *Let A be a family of algorithms, I a set of problem instances. Let cost be a cost metric applied to an algorithm-instance pair. Algorithm A^* is instance-optimal with respect to A and I for the cost metric cost if there exist constants k_1 and k_2 such that, for all $A \in A$ and $I \in I$:*

$$\text{cost}(A^*, I) \leq k_1 \text{cost}(A, I) + k_2$$

If A^* is instance-optimal, then any algorithm can improve with respect to A^* by only a constant factor r , known as the optimality ratio of A^* . Instance optimality is a much stronger notion than optimality in the average or worst case.

2.4 Ranking queries

Ranking queries, also known as top- k queries, focus on retrieving the k best answers from a potentially extensive result set based on the concept of relevance.

Algorithm The algorithm takes as inputs the cardinality k , the dataset R , and the scoring function S , with the output being the top k tuples with the highest scores according to S . The algorithm's approach involves computing scores for all tuples, sorting them based on these scores, and finally returning the top k tuples.

However, this straightforward method becomes expensive for large datasets, especially when multiple relations are involved, necessitating the sorting of significant amounts of data. Two crucial abilities are required:

- Ordering the tuples according to their scores with `ORDER BY`.
- Limiting the output cardinality to k tuples with `FETCH FIRST k ROWS ONLY`.

Example:

Consider the following queries:

```
-- Query one
SELECT *
FROM UsedCarsTable
WHERE Vehicle = 'Audi/A4'
AND Price <= 21000
ORDER BY 0.8*Price+0.2*Miles
-- Query two
SELECT *
FROM UsedCarsTable
WHERE Vehicle = 'Audi/A4'
ORDER BY 0.8*Price+0.2*Miles
```

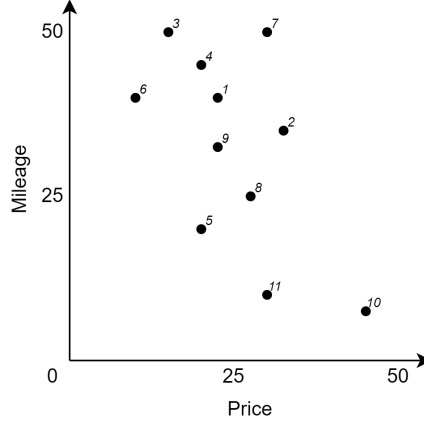
Here, the values 0.8 and 0.2, termed weights, normalize preferences on price and mileage. The first query might miss relevant answers (near-miss), could result in information overload by returning all Audi A4 cars in the dataset.

Only the initial k tuples are included in the result. If multiple sets of k tuples meet the ordering directive, any of these sets is considered a valid answer, demonstrating non-deterministic semantics.

Evaluation The evaluation of a top- k query considers two basic aspects: query type (one relation, many relations, and aggregate results) and access paths (no index, indexes on all/some ranking attributes). In the simplest case, in a top- k selection query with a single relation, if the input is sorted according to S , only the first k tuples need to be read. If the tuples are not sorted, an in-memory sort is performed with a complexity of $O(N \log k)$.

Example:

Consider the two-dimensional attribute space (Price, Mileage) illustrated in the previous example:



In this representation, each tuple corresponds to a two-dimensional point (p, m) , where p denotes the Price, and m represents the Mileage. The intuition behind minimizing $0.8 \cdot \text{Price} + 0.2 \cdot \text{Mileage}$ is the same as searching for points in proximity to $(0, 0)$. Our preferences play a crucial role in determining the outcome. Consider the equation of the line:

$$0.8 \cdot \text{Price} + 0.2 \cdot \text{Mileage} = v$$

Here, v is a constant. This equation can be alternatively expressed as:

$$\text{Mileage} = -4 \cdot \text{Price} + 5 \cdot v$$

From the previous equation we see that all the lines have a slope -4 . By definition, all points on the same line are equally good. Modifying the weights may lead to variations in the optimal choice. Importantly, the target of a query is not confined to $(0, 0)$; it can be any point $q = (q_1, q_2)$.

Distance functions In a general context, to assess the quality of a tuple t , we calculate its distance from the target point q . The desirability increases as the distance from q decreases. When examining the distances, the model can be characterized by the following components:

- An m -dimensional space $A = (A_1, A_2, \dots)$ comprising ranking attributes, where $m \geq 1$.
- A relation $R(A_1, A_2, \dots, B_1, B_2, \dots)$, with $B_1, B_2, \dots$ representing additional attributes.
- A target point $q = (q_1, q_2, \dots)$ located in A .
- A distance function $d : A \times A \rightarrow \mathbb{R}^+$, quantifying the distance between points in A .

Definition (Nearest neighbors). The top- k query is called k -nearest neighbors if given a point q , a relation R , an integer $k \geq 1$, and a distance function d it determines the k tuples in R that are closest to q according to d .

Definition (L_p -norms). The distance function, denoted as L_p -norms, is defined by the formula:

$$L_p(t, q) = \left(\sum_{i=1}^m |t_i - q_i|^p \right)^{\frac{1}{p}}$$

Euclidean distance	$L_2(t, q) = \sqrt{\sum_{i=1}^m t_i - q_i ^2}$
Manhattan distance	$L_1(t, q) = \sum_{i=1}^m t_i - q_i $
Čebyšëv distance	$L_\infty(t, q) = \max_i \{ t_i - q_i \}$

Table 2.1: Examples of L_p norms

The shape of the attribute space depends on the distance function and the weight associated to each coordinate.

Euclidean distance	$L_2(t, q; W) = \sqrt{\sum_{i=1}^m w_i t_i - q_i ^2}$
Manhattan distance	$L_1(t, q; W) = \sum_{i=1}^m w_i t_i - q_i $
Čebyšëv distance	$L_\infty(t, q; W) = \max_i \{w_i t_i - q_i \}$

Table 2.2: Examples of weighted L_p norms

Top-k join query In a top- k join query with $n > 1$ input relations and a scoring function S defined on the result of the join, the general formulation is as follows:

```

SELECT <attributes>
FROM R1,R2,...,Rn
WHERE <conditions>
ORDER BY S(p1,p2,...,pm) [DESC]
FETCH FIRST k ROWS ONLY

```

In this query, $p_1, p_2, \dots, p_m$ represent the scoring criteria. In top- k 1 – 1 join queries, where all joins are on a common key attribute, two main scenarios are possible:

- There is an index for retrieving tuples according to each preference.
- The relation is spread over several sites, each providing information only on part of the objects.

We make the following assumptions:

- Each input list supports sorted access, returning the identifier of the next best object and its partial score p_j .
- Each input list supports random access, returning the partial score of an object given its identifier.
- The identifier of an object is the same across all inputs.
- Each input consists of the same set of objects.

Example:

Given reviews on two different sites:

EatWell

Name	Score
The old mill	9.2
The canteen	9.0
Cheers!	8.3
Da Gino	7.5
Let's eat!	6.4
Chez Paul	5.5
Los pollos hermanos	5.0

BreadAndWine

Name	Score
Da Gino	9.0
Cheers!	8.5
The old mill	7.5
Chez Paul	7.5
The canteen	7.0
Los pollos hermanos	6.5
Let's eat!	6.0

If we aggregate the two tables with the following query:

```
SELECT *
FROM EatWell EW, BreadAndWine BW
WHERE EW.Name = BW.Name
ORDER BY EW.Score + BW.Score DESC
FETCH FIRST 1 ROW ONLY
```

The result would be:

Name	Global score
Cheers!	16.8
The old mill	16.7
Da Gino	16.5
The canteen	16.0
Chez Paul	13.0
Let's eat!	12.4
Los pollos hermanos	11.5

Note that the winner is not the best locally.

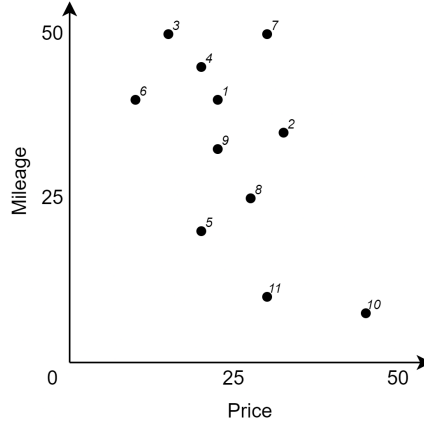
Score space Each object o returned by the input L_j has an associated local/partial score $p_j(o) \in [0, 1]$. Scores are normalized, and the hypercube $[0, 1]^m$ is called the score space. The point $p(o) = (p_1(o), p_2(o), \dots, p_m(o)) \in [0, 1]^m$ is the map of o into the score space. The global score $S(o)$ of o is computed using a scoring function S that combines the local scores of o :

$$S(o) \equiv S(p(o)) = S(p_1(o), p_2(o), \dots, p_m(o))$$

With $S : [0, 1]^m \rightarrow \mathbb{R}^+$.

Example:

Consider the attribute space $A = (\text{Price}, \text{Mileage})$

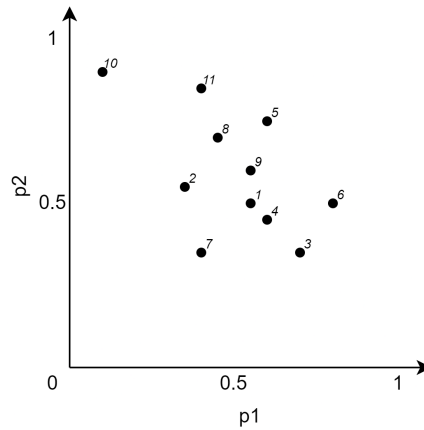


If we choose $\text{MaxP} = 50,000$ and $\text{MaxM} = 80,000$, a generic point o is mapped with these formulas:

$$p_1(o) = 1 - \frac{o.\text{Price}}{\text{MaxP}}$$

$$p_2(o) = 1 - \frac{o.\text{Mileage}}{\text{MaxM}}$$

The new graph is as follows:



Scoring functions The most common scoring functions are:

Average	$\text{SUM}(o) \equiv \text{SUM}(p(o)) = p_1(o) + p_2(o) + \dots + p_m(o)$
Weighted sum	$\text{WSUM}(o) \equiv \text{WSUM}(p(o)) = w_1p_1(o) + w_2p_2(o) + \dots + w_mp_m(o)$
Minimum	$\text{MIN}(o) \equiv \text{MIN}(p(o)) = \min\{p_1(o), p_2(o), \dots, p_m(o)\}$
Minimum	$\text{MAX}(o) \equiv \text{MAX}(p(o)) = \max\{p_1(o), p_2(o), \dots, p_m(o)\}$

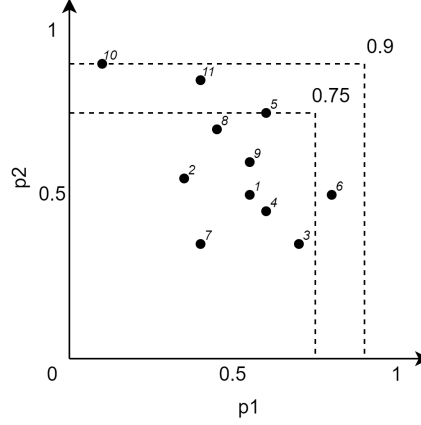
Table 2.3: Scoring functions

Even with the minimum scoring function, the goal is to retrieve the k objects with the highest global scores.

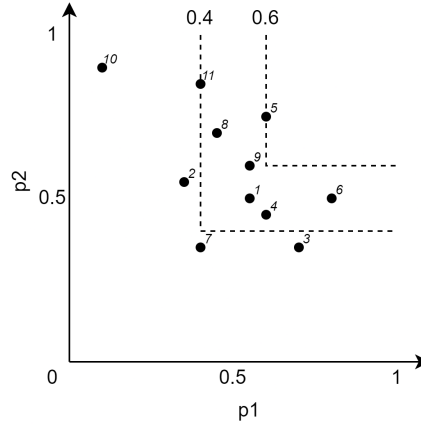
Iso-score curves Iso-score curves in the score space can be defined, representing sets of points with the same global score.

Example:

In the previous graph we have that the iso-score curves for the maximum function are:



In the previous graph we have that the iso-score curves for the minimum function are:



2.4.1 B-zero algorithm

Efficient computation of the result of a top- k 1-1 join query using a maximum scoring function S can be achieved through the B_0 algorithm. The inputs for this algorithm are an integer $k \geq 1$ and a ranked list $R_1, \dots, R_m$. The algorithm proceeds as follows:

1. Perform k sorted accesses on each list and store objects and partial scores in a buffer B .
2. For each object in B , compute the maximum of its (available) partial scores.
3. Return the top k objects with maximum score.

Property 2.4.1. B_0 algorithm is instance-optimal.

Example:

For instance, given reviews on two different sites:

EatWell

Name	Score
The old mill	9.2
The canteen	9.0
Cheers!	8.3
Da Gino	7.5
Let's eat!	6.4
Chez Paul	5.5
Los pollos hermanos	5.0

BreadAndWine

Name	Score
Da Gino	9.0
Cheers!	8.5
The old mill	7.5
Chez Paul	7.5
The canteen	7.0
Los pollos hermanos	6.5
Let's eat!	6.0

If we apply the B_0 algorithm with $k = 3$, selecting the first three rows from both tables (which are sorted), and summing all the values by identifier, we obtain the final rank with the maximum value:

Name	S
The old mill	9.2
Da Gino	9.0
The canteen	9.0
Cheers!	8.5

2.4.2 A-zero algorithm

The A-zero algorithm, also known as Fagin's algorithm, is applicable to every monotonic scoring function. The algorithm takes an integer k and a monotone function S combining ranked lists $R_1, \dots, R_m$ as inputs, producing the top k (object-score) tuples as output. The algorithm follows these steps:

1. Extract the same number of objects through sorted accesses in each list until there are at least k common objects.
2. For each extracted object, compute its overall score by making random accesses wherever needed.
3. Among these, output the k objects with the best overall score.

The complexity of this algorithm is $O(N^{\frac{m-1}{m}} k^{\frac{1}{m}})$, which is sublinear in the number N of objects. The stopping criterion is function-independent.

Property 2.4.2. The algorithm is not instance-optimal.

Example:

Consider hotel reviews on different sites:

Name	Cheapness	Name	Rating
Ibis	0.92	Crillon	0.9
Etap	0.91	Novotel	0.9
Novotel	0.85	Sheraton	0.8
Mercure	0.85	Hilton	0.7
Hilton	0.825	Ibis	0.7
Sheraton	0.8	Ritz	0.7
Crillon	0.75	Lutetia	0.6
...	...	...	...

Using the scoring function $0.5 \cdot \text{cheapness} + 0.5 \cdot \text{rating}$ and $k = 2$, iterate on the rows until k hotels are found in all columns (in this case, Ibis, Novotel, and Hilton). Complete the score of the remaining incomplete hotels by making random accesses to find the missing values. After summing up the information, compute the final ranking with the given score function, resulting in:

Top k	Score
Novotel	0.875
Crillon	0.825

Drawbacks The primary limitation of this algorithm lies in its dependence on accesses, leading to an underutilization of the specificity inherent in certain scoring functions. Additionally, the memory requirements have the potential to become impractical. While there are possibilities for improvements, substantial enhancements in complexity can only be achieved by modifying the stopping condition.

2.4.3 Threshold algorithm

The threshold algorithm takes as inputs an integer k and a monotone function S that combines ranked lists $R_1, \dots, R_m$. It outputs the top k (object, score) tuples. The algorithm follows these steps:

1. Perform sorted access in parallel in each list R_i .
2. For each object o , conduct random accesses in the other lists R_j to extract score s_j .
3. Compute the overall score $S(s_1, \dots, s_m)$. If the value is among the k highest seen so far, and remember o .
4. Let s_{L_i} be the last score seen under sorted access for R_i .
5. Define the threshold $T = S(s_{L_1}, \dots, s_{L_m})$.
6. If the score of the k -th object is worse than T , go to step one.
7. Return the current top k objects.

The stopping criterion depends on the function and not on the number of accesses.

Property 2.4.3. The threshold algorithm is instance optimal among all algorithms that use random and sorted accesses.

Example:

Consider hotel reviews with cheapness and rating values.

Name	Cheapness	Name	Rating
Ibis	0.92	Crillon	0.9
Etap	0.91	Novotel	0.9
Novotel	0.85	Sheraton	0.8
Mercure	0.85	Hilton	0.7
Hilton	0.825	Ibis	0.7
Sheraton	0.8	Ritz	0.7
Crillon	0.75	Lutetia	0.6
...	...	...	...

The scoring function is $0.5 \cdot \text{cheapness} + 0.5 \cdot \text{rating}$, and $k = 2$. We do a sorted access, and we put the hotels in the first row in the buffer with the mean value of both variables (the second can be found with a random access). The threshold point is $(0.92, 0.9)$ and has a value of 0.91, found with the scoring function. We continue to do this procedure, and we update the top k hotels only if the found hotel has a better score of at least one of the hotels in the buffer (note that we keep in the buffer only the top k hotels, while we delete the others). We stop the iteration when the value of all the objects in the buffer is greater or equal to the threshold value. The buffer contains the top k hotels with their corresponding scores.

Top k	Score
Novotel	0.875
Crillon	0.825

The threshold point has a value of 0.825.

Performance evaluation In general, the Threshold Algorithm outperforms the Fagin Algorithm since it can adapt to the specific scoring function. To assess the performance of Threshold Algorithm, we evaluate the middleware cost using the following formula:

$$\text{cost} = SA \cdot c_{SA} + RA \cdot c_{RA}$$

Where:

- SA is the total number of sorted accesses.
- RA is the total number of random accesses.
- c_{SA} is the base cost of sorted accesses.
- c_{RA} is the base cost of random accesses.

2.4.4 No Random Access algorithm

The No Random Access algorithm returns the top- k objects, but their scores might be uncertain. The fundamental idea behind this algorithm is to maintain, for each selected object o , both a lower bound and an upper bound on its score. Thus, an unlimited buffer is required to store the

objects, sorted by decreasing lower bound values. The algorithm stops when the upper bound of a new object is less than or equal to the worst lower bound among the best objects. The inputs include an integer $k \geq 1$, a monotone function S combining ranked lists $R_1, \dots, R_m$, and the output is the ranking without the scores of the objects. The procedure is as follows:

1. Perform a sorted access to each list.
2. Store each retrieved object o in buffer B , maintaining $S^-(o)$ and $S^+(o)$ along with a threshold τ .
3. Repeat from step one as long as:

$$S^-(B[k]) < \max\{\max\{S^+(B[i]), i > k\}, S(\tau)\}$$

Example:

Given tables based on three different criteria:

R_1		R_2		R_3	
ID	Score	ID	Score	ID	Score
o_1	1.0	o_2	0.8	o_7	0.6
o_7	0.9	o_3	0.75	o_2	0.6
o_2	0.7	o_4	0.5	o_3	0.5
o_6	0.2	o_1	0.4	o_5	0.1
...	...	...	...	...	...

We decide to use sum as score function, and $k = 2$. For the first iteration we add to the buffer the objects in the first row with a lower bound corresponding to the sum of the score of each object and the upper bound corresponding to the sum of all the object in the row. The threshold is the sum of all the scores of the row:

Identifier	Lower bound	Upper bound
o_1	1.0	2.4
o_2	0.8	2.4
o_7	0.6	2.4

The threshold has a value of 2.4, and so we have that:

$$0.8 < \max\{2.4, 2.4\}$$

Since that the inequality is verified we have to do another iteration, that creates the following table:

Identifier	Lower bound	Upper bound
o_7	1.5	2.25
o_2	1.4	2.3
o_1	1.0	2.35
o_3	0.75	2.25

The threshold has a value of 2.25, and so we have that:

$$1.4 < \max\{2.35, 2.25\}$$

Since that the inequality is verified we have to do another iteration, that creates the following table:

Identifier	Lower bound	Upper bound
o_2	2.1	2.1
o_7	1.5	2.0
o_3	1.25	1.95
o_1	1.0	2.0
o_4	0.5	1.7

The threshold has a value of 1.7, and so we have that:

$$1.5 < \max\{2.0, 1.7\}$$

Since that the inequality is verified we have to do another iteration, that creates the following table:

Identifier	Lower bound	Upper bound
o_2	2.1	2.1
o_7	1.5	1.9
o_1	1.4	1.5
o_3	1.25	1.45
o_4	0.5	0.8
o_6	0.2	0.7
o_5	0.1	0.7

The threshold has a value of 1.7, and so we have that:

$$1.5 < \max\{1.5, 0.7\}$$

Since that the inequality is not verified the algorithm return the first two elements in the table.

Property 2.4.4. No Random Access algorithm is instance optimal among all algorithms that do not make random accesses.

2.4.5 Summary

Algorithm	Scoring function	Data access	Notes
B_0	MAX	sorted	instance-optimal
FA	monotone	sorted and random	cost independent of function
TA	monotone	sorted and random	instance-optimal
NRA	monotone	sorted	instance-optimal, uncertain

The main aspects of the ranking queries are:

- Effectiveness in pinpointing the top objects based on a designated scoring function.
- Exceptional control over the result's cardinality.

- High computational efficiency.
- Ease of expressing the relative importance of attributes.
- Challenges for users in specifying a scoring function.

2.5 Skyline queries

Skyline queries aim to identify superior objects across multiple perspectives, relying on the concept of dominance.

Definition (*Domination*). A tuple t dominates a tuple s ($t \prec s$) if and only if t is nowhere worse than s :

$$\forall i, 1 \leq i \leq m \rightarrow t[A_i] \leq s[A_i]$$

And is better at least once:

$$\exists j, 1 \leq j \leq m \wedge t[A_j] < s[A_j]$$

Definition (*Skyline*). The skyline of a relation is the set of its non-dominated tuples.

The convention on the skyline queries is that lower values are better than higher values. The convention is that lower values are considered better. A tuple is part of the skyline if it is the top-1 result with respect to at least one monotone scoring function.

Query syntax The syntax for skyline queries can be expressed as follows:

```
SELECT <attributes>
FROM R1,R2,...,Rn
WHERE <conditions>
GROUP BY <conditions>
HAVING <conditions>
SKYLINE OF [DISTINCT] d1[MIN|MAX|DIFF], ..., dm[MIN|MAX|DIFF]
ORDER BY <conditions>
```

This query can be easily translated into a standard query, but the result is too slow and cannot be used in practice.

2.5.1 Block Nested Loop algorithm

The Block Nested Loop algorithm takes a dataset D of multidimensional points as input and outputs the skyline of D . Its complexity is $O(n^2)$, making it inefficient for large datasets.

Algorithm 1 Block nested loop algorithm

```
1:  $W \leftarrow \emptyset$ 
2: for every point  $p$  in  $D$  do
3:   if  $p$  not dominated by any point in  $W$  then
4:     remove from  $W$  the points dominated by  $p$ 
5:     add  $p$  to  $W$ 
6:   end if
7: end for
8: return  $W$ 
```

2.5.2 Sort Filter Skyline algorithm

The Sort Filter Skyline algorithm also takes a dataset D of multidimensional points as input and outputs the skyline of D . It performs better than the Block Nested Loop algorithm, especially for large datasets, but still has a complexity of $O(n^2)$.

Algorithm 2 Sort filter skyline algorithm

```

1:  $S \leftarrow D$ 
2:  $W \leftarrow \emptyset$ 
3: for every point  $p$  in  $S$  do
4:   if  $p$  not dominated by any point in  $W$  then
5:     add  $p$  to  $W$ 
6:   end if
7: end for
8: return  $W$ 

```

Example:

Consider the following unordered dataset:

Name	Cost	Complaints
Crillon	0.25	0.1
Ibis	0.08	0.3
Hilton	0.175	0.3
Sheraton	0.2	0.2
Novotel	0.15	0.1

The skyline, consisting of Novotel and Ibis, is identified by the algorithm.

2.5.3 Summary

Skyline queries are effective in identifying potentially interesting objects when user preferences are unknown. They are simple to use but return a large number of objects, and their computation is not highly efficient. Skyline queries are agnostic with respect to user preferences.

K-skyband To enhance skyline queries, the concept of k -skyband is introduced, where the result includes a set of tuples dominated by fewer than k tuples.

2.6 Comparison between ranking and skyline

	Ranking queries	Skyline queries
Simplicity	No	Yes
Only interesting results	No	Yes
Control of cardinality	Yes	No
Trade-off among attributes	Yes	No

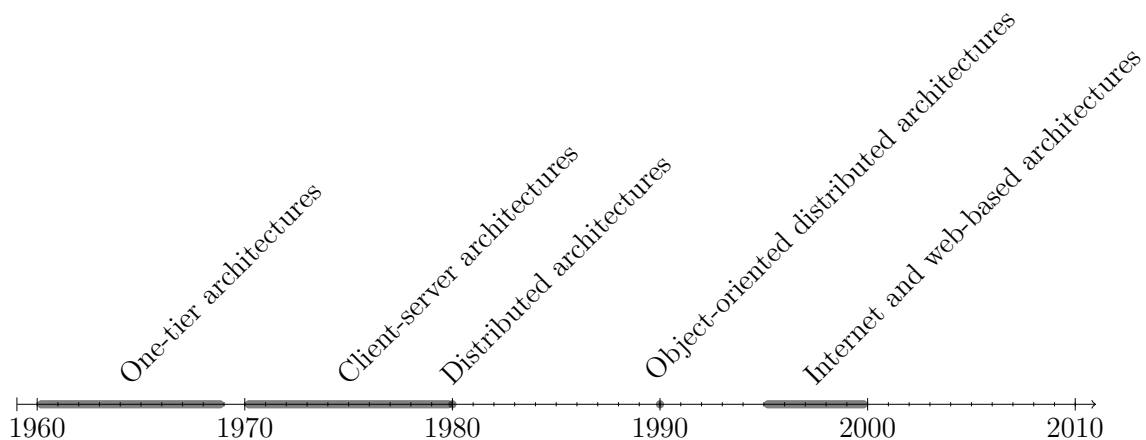
CHAPTER 3

Java Enterprise Edition

3.1 Introduction

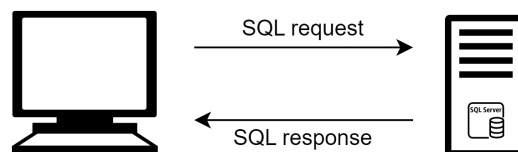
Definition (*Architecture*). Architecture refers to the comprehensive integration of hardware, software, and network resources.

Examples of contemporary architectures include distributed web-based and mobile architectures, service-oriented architectures, cloud and virtualized architectures, and application service provisioning architectures.



3.2 Client-server architecture

In the client-server architecture, hardware components include a server responsible for data management and a client focused on presentation layout.



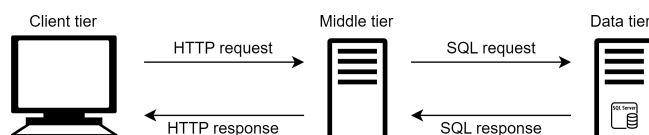
The client software issues requests to the server via SQL queries and handles both business logic and presentation logic. The server software processes these queries and sends the resulting

data back to the client, focusing on data management.

The network topology is typically organized as a Local Area Network (LAN) with one or more servers and multiple clients.

3.3 Three-tier architecture

The three-tier architecture comprises a server for data management, a client for presentation layout, and a middle tier to enhance the separation between the client and the server.

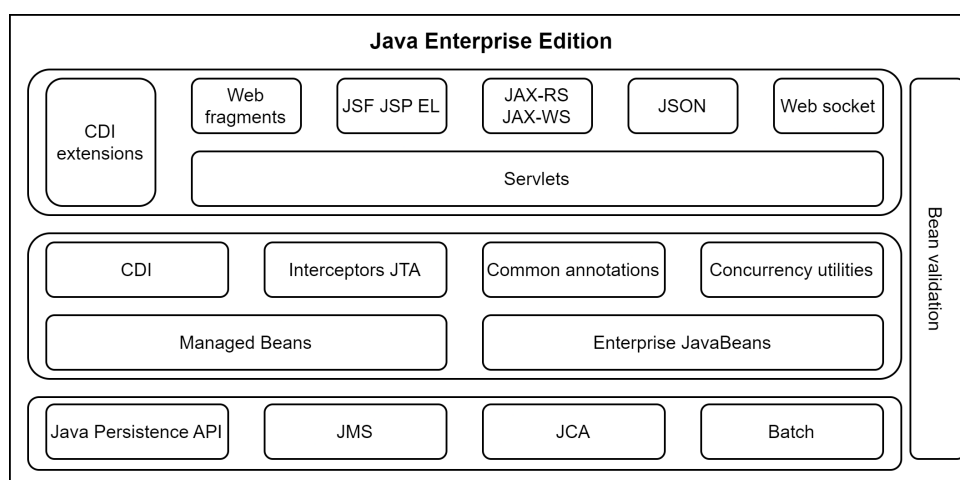


This architecture has several variants, depending on the software features of the middle tier:

- *Web Pure HTML*: the client functions as a standard web browser, focusing solely on the presentation layout (thin client). The middle tier incorporates a web server utilizing HTTP, hosts the business logic for dynamically generating content from the raw data of the data tier, and manages the presentation layout.
- *Rich Internet Application (RIA)*: this is a fusion of web and desktop applications using JavaScript. The client is termed fat due to its standard communication protocol, language, and API. Noteworthy features of this architecture include novel interface event types, including those specific to touch and mobile apps, asynchronous interaction (AJAX), client-side persistent data, offline application capabilities, and native multimedia and 3D support.

3.4 Java Enterprise Edition

Java Enterprise Edition (JEE) is a platform designed for the creation, deployment, and maintenance of three-tier web applications.



JEE comprises the following elements:

1. *Java Database Connectivity* (JDBC): this is the initial industry standard for establishing database-independent connectivity between Java and databases. JDBC allows establishing connections with databases, sending SQL statements, and processing the results.
2. *Servlets*: these are Java technologies used in the presentation tier of web applications. Servlets offer a platform-independent, component-based approach for constructing web-based applications. They have access to other Java APIs for interfacing with enterprise databases and operate within a container for concurrency control and lifecycle management.
3. *Enterprise Java Beans* (EJB): EJB enables the development of distributed, transactional, secure, and portable applications. EJB components facilitate interaction between the web front-end and business functions, as well as data access services.
4. *Java Persistence API* (JPA): JPA defines an interface for mapping relational data to objects in Java.
5. *Java Transaction API* (JTA): JTA manages transactions in Java, allowing components to initiate, commit, and roll back transactions in a resource-agnostic manner. With JTA, Java components can oversee multiple resources within a single transaction using a unified interaction model.

3.5 Java Persistence API

The technique of bridging the gap between the object model and the relational model is known as Object Relational Mapping (ORM).

Definition (*Impedance mismatch*). The challenge of mapping one model to the other lies in the concepts in one model for which there is no logical equivalent in the other, referred to as impedance mismatch.

The automatic transformation of a model into another is managed by an element called a mediator. The principal distinctions between the object-oriented model and the relational one are outlined below:

Object-oriented model	Relational model
Objects, classes	Tables, rows
Attributes, properties	Columns
Identity (physical memory address)	Primary key
Reference to other entity	Foreign key
Inheritance/Polymorphism	Not supported
Methods	Stored procedures, triggers
Code is portable	Not necessarily portable

The JPA addresses the gap between object-oriented domain models and relational database systems by utilizing Plain Old Java Objects (POJO), providing a persistence model for ORM.

Features Key features of the JPA include:

- *POJO persistence*: objects being persisted need not possess any special characteristics; any existing non-final object with a default constructor can be persisted.
- *Non-intrusiveness*: the persistence API exists as a separate layer from the persistent objects.
- *Object queries*: a powerful query framework offers the ability to query across entities and their relationships without having to use concrete foreign keys or database columns.

Definition (*Entity*). An entity is a class representing a collection of persistent objects mapped onto a relational table.

Definition (*Persistence unit*). The persistence unit is the set of all classes that are persistently mapped to one database.

Definition (*Persistence context*). The persistence context is the set of all managed objects of the entities defined in the persistence unit.

Definition (*Managed entity*). A managed entity is an entity part of a persistence context for which the changes of the state are tracked.

Definition (*Entity Manager*). The Entity Manager is the interface for interacting with a persistence context.

Definition (*Client*). The client is a component that can interact with a persistence context, indirectly through an Entity Manager.

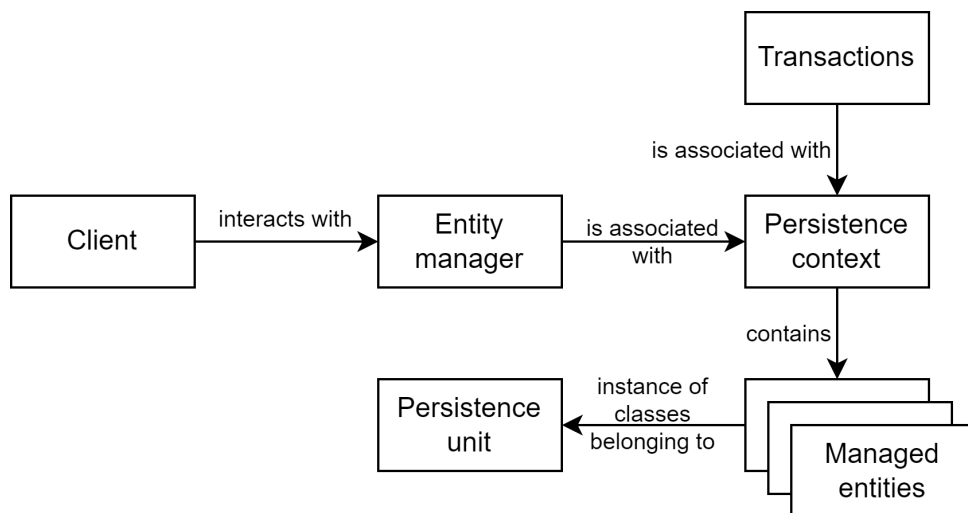


Figure 3.1: JPA structure

Entities are accessed through the Entity Manager interface of the JPA. The Entity Manager exposes all the operations needed to synchronize the managed entities in the persistence context to the database:

- Persist an entity instance in the database.
- Find an entity instance by its primary key.

- Remove an entity instance from the database.
- Reset the entity instance from the database.
- Write the state of entities to the database immediately.

Entities An entity is a EJB associated with a tuple in a database. The persistent counterpart of an entity has a lifespan longer than that of the application. The entity class must be associated with the database table it represents. An entity can enter a managed state, where all modifications to the object's state are tracked and automatically synchronized to the database. Entities must meet certain requirements:

- Must have a public or protected constructor with no arguments.
- Must not be `final`.
- No method or persistent instance variables may be `final`.
- The `Serializable` interface must be implemented if you pass the entity by value.

In the database, objects and tuples have an identity (primary key), so an entity assumes the identity of the persistent data it is associated with. The primary key can be either simple or composite. To identify a primary key, we use the `@Id` annotation, and for composite keys, we use `@EmbeddedId` and `@IdClass` annotations.

Data uniqueness At times, applications may prefer not to handle the explicit management of data value uniqueness. In such scenarios, the persistence provider can automatically create an identifier for each entity instance of a specified type. This functionality, known as identifier generation, is defined by the `@GeneratedValue` annotation. There are four available ID generation strategies that applications can opt for:

1. **Auto.**
2. **Table:** identifiers are generated based on a generator table.
3. **Sequence:** the identifiers are generated using sequences.
4. **Identity:** the identifiers are generated using primary keys identity columns.

Example:

An identifier can be generated as follows:

```
@Entity
public class Mission implements Serializable {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;
    private String city;
}
```

Attributes can be qualified with properties that guide the mapping between POJO and relational tables. These properties include large objects, enumerated types, and temporal types.

Fetch policies The fetch policy can be either lazy (retrieve the item when needed) or eager (retrieve the item as soon as possible). Lazy policy is primarily employed for large objects.

Example:

The qualifiers can be used as follows:

```
@Entity
public class Mission implements Serializable {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    // date
    @Temporal(TemporalType.DATE)
    private Date date;

    // image
    @Basic(fetch=FetchType.LAZY)
    @LOB
    private byte[] photo;
}
```

Entities are ordinarily mapped to tables with corresponding names, and their fields are mapped to columns with identical names by default. However, it is possible to alter this behavior using certain annotations. In cases where an entity should not be persisted, it can be designated with the `@Transient` annotation.

Example:

The mapping can be redefined as follows:

```
@Entity @Table(name="T_BOOKS")
public class Book {
    @Column(name="BOOK_TITLE", nullable=false)
    private String title;
    private CoverType coverType;
    private Date publicationDate;

    // non persistent entity
    @Transient
    private BigDecimal discount;
}
```

3.5.1 Mapping

In any object model, relationships exhibit four key characteristics:

- *Directionality*: each of the two entities involved may possess an attribute that facilitates access to the other entity. If two entities are mutually connected, it forms a bidirectional relationship; otherwise, it constitutes a unidirectional relationship.

- *Role*: each entity within the relationship plays a role in one direction of access. Entities are categorized as source and target based on the relationship's direction.
- *Cardinality*: this refers to the number of instances of entities on each side of the relationship. Four cardinality possibilities exist:
 - *Many-to-one*: many source entities to one target entity.
 - *One-to-many*: one source entity to many target entities.
 - *One-to-one*: one source entity to one target entity.
 - *Many-to-many*: many source entities to many target entities.
- *Ownership*: in a relationship, one of the two entities is deemed the owner. In the database, relationships are implemented through a foreign key column (referred to as the join column in JPA) that points to the key of the referenced table. The entity with a foreign key column is considered the owner of the relationship, and its side is termed the owning side.

One-to-many relationship The bidirectional one-to-many relationship is established using the `@ManyToOne` and `@OneToMany` annotations. In the entity in relationships one to many, we use:

- The `@ManyToOne` annotation is applied to the entity participating with multiple instances, designating the entity containing this annotation as the owner of the relationship.
- The `@JoinColumn` annotation is utilized to specify the foreign key column of the underlying table.

Example:

Here's the first part of the definition for a one-to-many bidirectional mapping:

```
@Entity
public class Employee {
    @Id private int id;
    @ManyToOne
    @JoinColumn(name="dept_fk")
    private Department dept;
}
```

To achieve bi-directionality, the mapping direction for the one-to-many relationship must also be specified. This is accomplished by including a `@OneToMany` annotation in the entity participating with one instance. The `@OneToMany` annotation is placed on a collection data member and includes a `mappedBy` element to indicate the property implementing the inverse of the relationship.

Example:

Here's the second part of the definition for a one-to-many bidirectional mapping:

```
@Entity
public class Department {
    @Id private int id;
```

```
@OneToMany(mappedBy="dept")
private Collection<Employee> employees;
}
```

Many-to-one relationship The many-to-one relationship is defined similarly to the one-to-many relationship, but with the source and target switched.

One-to-one relationship To establish a one-to-one relationship in JPA, you can choose between two approaches:

- Map the relationship similarly to the bidirectional case, but only use the one-to-many direction.
- Omit the mapping of the collection attribute in the entity participating with one instance, and instead, use a query to retrieve related instances, relying on the inverse (many-to-one) relationship mapping.

In a one-to-one mapping, either entity can be the owner, depending on the database design. A one-to-one mapping is declared by annotating the owner entity with the `@OneToOne` annotation.

Example:

Here's the first part of the definition for a one-to-one mapping:

```
@Entity
public class Employee {
    @Id private int id;
    @OneToOne
    private ParkingSpace parkingSpace;
}
```

For a bidirectional one-to-one mapping, the inverse side of the relationship must also be specified. In the non-owner entity, both the `@OneToOne` annotation and the `mappedBy` element are needed to guide JPA in placing the foreign key.

Example:

Here's the second part of the definition for a one-to-one mapping:

```
@Entity
public class ParkingSpace {
    @Id private int id;
    @OneToOne(mappedBy="parkingSpace")
    private Employee employee;
}
```

Many-to-many relationship In a many-to-many mapping, there is no direct foreign key column; instead, a join table is used. Either entity can be designated as the owner. The logical model of a many-to-many relationship requires a bridge table (join table in JPA).

Example:

Here's an example specifying the mapping of the entity to the bridge table via annotations:

```
@Entity
public class Employee {
    @Id private long id;
    private String name;
    @ManyToMany
    @JoinTable(name="EMP_PROJ",
               joinColumns=@JoinColumn(name="EMP_ID"),
               inverseJoinColumns=@JoinColumn(name="PROJ_ID"))
    private Collection<Project> projects;
}
```

For a bidirectional many-to-many mapping, the inverse side of the relationship must also be specified. In the non-owner entity, both the `@ManyToMany` annotation and the `mappedBy` element are required.

Example:

Here's the second part of the definition for a many-to-many mapping:

```
@Entity
public class Project {
    @Id private int id;
    @ManyToMany(mappedBy="projects")
    private Collection<Employee> employees;
}
```

3.5.2 Relationship fetch mode

When the fetch mode is unspecified, the default behavior is as follows:

- Single-valued relationships are fetched eagerly.
- Collection-valued relationships are loaded lazily.

In bidirectional relationships, the fetch mode may be lazy on one side but eager on the other. It's recommended to consider lazy loading as the most suitable mode for all relationships to avoid performance drawbacks from eager loading of unused relationships.

Example:

Here's an example using the annotation to define lazy loading:

```
@Entity
public class Employee {
    @Id private int id;
    @OneToOne(fetch=FetchType.LAZY)
    private ParkingSpace parkingSpace;
}
```

The directive to lazily fetch an attribute serves as a hint to the persistence provider, which can still employ an eager loading policy if necessary. However, an eager policy cannot be replaced with a lazy one by the provider.

3.5.3 Cascading operations

By default, operations performed by the Entity Manager do not cascade to other entities in a relationship with the operated entity. However, in certain cases, it may be desirable to propagate changes to related entities, which can be achieved by enabling manual cascading.

Example:

Activation of manual cascading of type persist:

```
@Entity
public class Employee {
    @ManyToOne(cascade=CascadeType.PERSIST)
    Address address;
}
```

Cascade settings are unidirectional and must be set on both sides for bidirectional behavior.

Cascade remove JPA supports an additional remove cascading option called **ORPHANREMOVAL**. This is used in **@OneToOne** and **@OneToMany** annotations for privately owned parent-child relationships where each child entity is associated only with one parent entity through just one relationship. This operation causes the child entity to be removed when the parent-child relationship is broken by removing the parent, setting the attribute holding the related entity to null, or, in the one-to-many case, by removing the child entity from the parent's collection. The distinction between the **CascadeType.REMOVE** attribute and the **ORPHANREMOVAL** mode is that if the value of an entity is manually set to null, only **ORPHANREMOVAL** will automatically remove the linked entities from the database.

3.6 JPA Entity Manager

In JPA, entity instances are POJOs that only become managed when an API method is invoked to start the process. The Entity Manager is the central authority for all persistence actions, handling the mapping and APIs for database interactions.

Definition (*Persistence context*). The persistence context is a core concept in JPA, acting like a main memory database that holds objects in a managed state.

A managed object is tracked, enabling automatic synchronization of its state modifications with the database. Database writes generally occur asynchronously, requiring the persistence context to connect to a transaction when they happen. A managed entity has dual lives: one as a Java object and one as a relational tuple, existing only within the persistence context. When the POJO exits the persistence context, the binding breaks, and it is no longer tracked or synchronized with the database. Applications interact with the Entity Manager, not directly with the persistence context.

3.6.1 Operations

When an entity is first instantiated, it is in a transient state because the Entity Manager is not aware of its existence. Transient entities are not part of the persistence context associated with the Entity Manager.

Example:

A new POJO can be created as follows:

```
Employee emp = new Employee(ID, "John Doe");
```

To transition transient entities to the managed state, the `persist()` method of the Entity Manager is used. Once an entity becomes managed, any changes made to it are automatically applied to the database. The managed entity and its corresponding tuple are associated until the entity exits the managed state. You can call `persist()` on an already managed entity to trigger the cascade process.

Example:

A new POJO can be created and later made managed as follows:

```
Employee emp = new Employee(ID, "John Doe");  
em.persist(emp);
```

Entities can be retrieved using the `find()` method, which takes the class of the sought entity and the primary key value identifying the desired entity instance. If the entity instance is found, the returned object becomes managed; otherwise, the `find()` method returns null.

Example:

Finding an entity:

```
Employee emp = em.find(Employee.class, ID);
```

To remove an entity, the `remove()` method is used. This action breaks the association between the entity and the persistence context. Upon the transaction associated with the Entity Manager's persistence context committing, or when the `flush()` method of the Entity Manager is called, the tuple linked with the entity is scheduled for deletion from the database. Although the entity still exists, its changes are no longer tracked for synchronization with the database.

Example:

Removing an entity:

```
em.remove(emp);
```

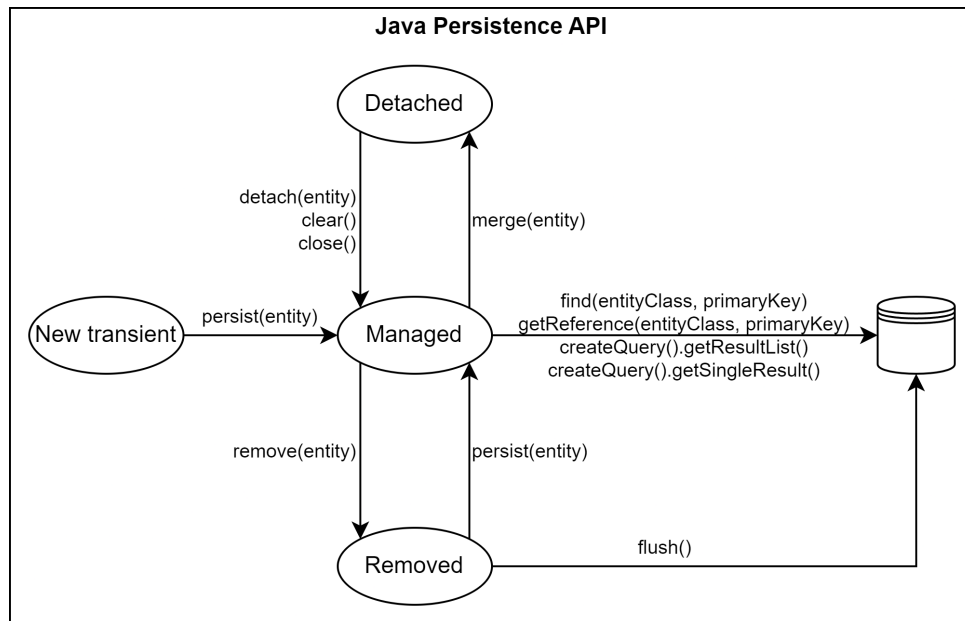


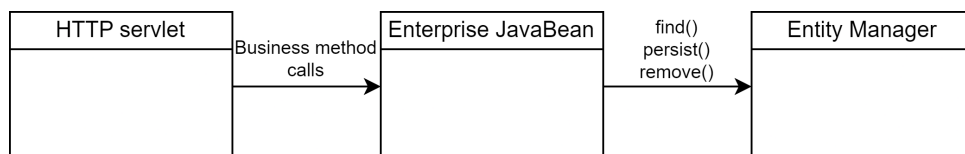
Figure 3.2: Possible states for the entities

Referring to the preceding diagram, entities can exist in five primary states:

- *New*: the entity is unknown to the Entity Manager, lacks a persistent identity, and has no associated tuple.
- *Managed*: the entity is associated with the persistence context, and changes to objects are automatically synchronized with the database.
- *Detached*: the entity has an identity potentially associated with a database tuple, but changes are not automatically propagated to the database.
- *Removed*: the entity is scheduled for removal from the database.
- *Deleted*: the entity is erased from the database.

3.6.2 Application architecture

In JEE, clients leverage the services of the EJB container to establish a connection with the Entity Manager. Specifically, the business layer interacts with the Entity Manager. The advantage of using EJB is that the container provides support to make JPA entity method calls transactional through the automatic creation of transactions.



Transactions in this context exist at three levels:

- *DBMS transactions*: managed by the DBMS and utilize SQL. This level is used by the Entity Manager.

- *Resource-local transactions*: managed by the application and use the JDBC connection interface, mapped to JDBC.
- *Container transaction*: managed by the application or the container and use the JTA interface, also mapped to JDBC. The Entity Manager operates at this level.

After defining a business object, the container injects the Entity Manager into it. The container transparently manages instances of the Entity Manager for the application. It provides the transaction required to save modifications made to the entities in the persistence context associated with the Entity Manager into the database.

Example:

Definition of a business object EJB:

```
@Stateless
public class myEJBService {
    @PersistenceContext(unitName = "MyPersistenceUnit")
    private EntityManager em;
}
```

When a client calls a method of a business object that utilizes a container-managed entity manager for persistence, the container offers a transaction for saving modifications to the database. If the same transaction is called multiple times, it is reused. This behavior is the most common and default, but the business object methods can be annotated to specify a different transactional behavior using `@TransactionAttribute(TransactionAttributeType.type)`, where `type` can be:

- **Mandatory**: expects a transaction to have already been started and be active when the method is called. If no transaction is active, an exception is thrown.
- **Required**: the default behavior; starts a new transaction if none is active. If one is active, it is used.
- **Requires_new**: the method always needs to be in its own transaction. Any active transaction is suspended.
- **Supports**: the method does not access transactional resources but tolerates running inside one if it exists.
- **Not_supported**: the method will cause the container to suspend the current transaction if one is active when the method is called.
- **Never**: the method will cause the container to throw an exception if a transaction is active when the method is called.

3.7 Summary

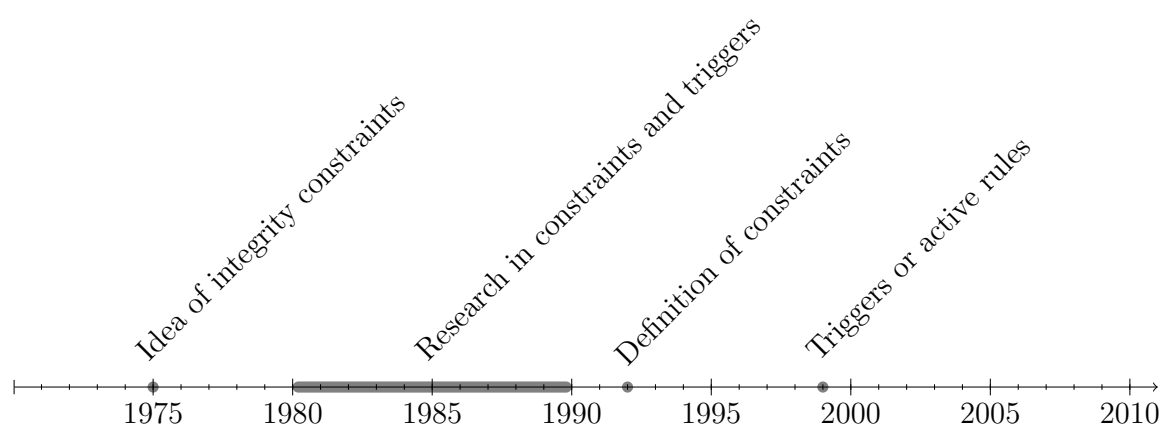
JPA offers several advantages. It eliminates the need to write SQL code, allowing developers to focus on business logic without worrying about database specifics. Application code can ignore table names, as the mapping is expressed through annotations with defaults. Additionally, there is no need to manually create or destroy database connections or manage transactions; these

tasks are handled automatically. Business object methods are inherently transactional, providing a default transactional behavior, with annotations available to specify different transaction management policies as needed.

Triggers

4.1 Definition and history

Definition (*Trigger*). A trigger refers to a set of actions that are automatically executed when an INSERT, UPDATE, or DELETE operation is carried out on a designated table.



Triggers are utilized to transition from a passive state to an active state, enabling automated responses to database changes.

4.2 Introduction

Triggers are built on the Event-Condition-Action paradigm, where an action A is automatically executed whenever a specific event E occurs, contingent upon the truth of a condition C. To be more specific, the components of triggers can be described as follows:

- *Event*: typically, this pertains to a change in the database's status, encompassing operations like insertions, deletions, and updates.
- *Condition*: this is a predicate that defines the specific circumstances in which the trigger's action must be executed.
- *Action*: the action entails a general update statement or a stored procedure, typically involving database modifications such as insertions, deletions, and updates. It may also encompass error notifications.

Triggers operate alongside integrity constraints and provide the capability to assess intricate conditions. These triggers are compiled and stored within the DBMS much like stored procedures. However, unlike stored procedures that are invoked by the client, triggers execute automatically in response to predefined events.

4.3 Triggers definition

The typical syntax of a trigger is as follows:

```
CREATE TRIGGER <TriggerName>
{BEFORE|AFTER}
{INSERT|DELETE|UPDATE [OF <ColumnName>]} ON <TableName>
REFERENCING {
    [OLD TABLE [AS] <OldTableAlias>]
    [NEW TABLE [AS] <NewTableAlias>] |
    [OLD [ROW] [AS] <OldTupleName>]
    [NEW [ROW] [AS] <NewTupleName>]
}
[FOR EACH {ROW|STATEMENT}]
[WHEN <Condition>]
<SQLProceduralStatement>
```

Modes Triggers can be executed in different modes, including:

- **BEFORE:** in this mode, the trigger's action occurs before any changes to the database's status, but only if the specified condition is met. It is primarily used for validating modifications before they are applied, potentially allowing for conditional effects. A significant constraint is that before triggers cannot directly update the database.
- **AFTER:** this mode involves executing the trigger's action after the database has undergone modifications, contingent upon the satisfaction of the specified condition. It is the most commonly used mode and is well-suited for a wide range of applications.

Granularity Triggers can function at various levels of granularity:

- *Row-level granularity:* in this approach, the trigger is evaluated and potentially executed separately for each tuple affected by the triggering statement. Writing triggers at the row-level is more straightforward but may be less efficient.
- *Statement-level granularity:* triggers are assessed and potentially executed only once for each activating statement, regardless of the number of tuples impacted in the target table. This occurs even if no tuple is affected. This mode aligns more closely with the traditional SQL statement approach, which is typically set-oriented.

Definition (*Transition variables*). In the context of triggers, transition variables are special variables that indicate the state of a modification, both **BEFORE** and **AFTER**.

The syntax depends on the level of granularity:

- Row-level: in this case, the variables old and new are used, where old represents the value before the modification of the specific row (tuple) being considered, and new represents the value after the modification.
- Statement-level: for this granularity, table variables are employed. These include old table and new table, with old table containing the old values of all affected rows and new table containing the new values of those rows.

It's important to note that, in specific cases, these variables may be undefined. In triggers with an event of INSERT, the variables old and old table are undefined, while in triggers with an event of DELETE, the variables new and new table are undefined.

Example:

Table T2 serves as a replica of table T1. Whenever an update is made to T1, this modification is automatically replicated in T2. This replication process is accomplished using triggers. The insertion of a new tuple is done with the following trigger:

```
CREATE TRIGGER REPLIC_INS
AFTER INSERT ON T1
FOR EACH ROW
INSERT INTO T2 VALUES (NEW.ID, NEW.VALUE);
```

The deletion of a tuple is done with the following trigger:

```
CREATE TRIGGER REPLIC_DEL
AFTER DELETE ON T1
FOR EACH ROW
DELETE FROM T2 WHERE T2.ID = OLD.ID;
```

The update of only the value a tuple is done with the following trigger:

```
CREATE TRIGGER REPLIC_UPD
AFTER UPDATE OF VALUE ON T1
WHEN NEW.ID = OLD.ID
FOR EACH ROW
UPDATE T2 SET T2.VALUE = NEW.VALUE
WHERE T2.ID = OLD.ID;
```

The previously mentioned trigger will not activate when the ID of a tuple is modified. Nonetheless, for a comprehensive and robust implementation, this scenario must also be taken into account.

If we want to replicate the rows only for the tuples whose value is ≥ 10 , we will have the following triggers:

```
-- Insertion
CREATE TRIGGER CON_REPL_INS
AFTER INSERT ON T1
FOR EACH ROW
WHEN (new.VALUE >= 10)
INSERT INTO T2 VALUES (new.ID, new.VALUE);
-- Deletion
CREATE TRIGGER Cond_REPL_DEL
AFTER DELETE ON T1
FOR EACH ROW
```

```
WHEN (old.VALUE >= 10)
DELETE FROM T2 WHERE T2.ID = old.ID;
```

Execution order When multiple triggers are linked to a common event, SQL:1999 dictates the sequence in which they execute:

1. BEFORE statement level triggers.
2. BEFORE row level triggers.
3. Modification is applied, and integrity constraints are checked.
4. AFTER row level triggers.
5. AFTER statement level triggers.

However, if there are multiple triggers within the same category, the order of execution is contingent upon the specific system's implementation.

Cascading The action of a trigger can lead to the activation of another trigger, a phenomenon known as cascading or nesting. Recursive cascading occurs when a statement, S, executed on table T initiates a sequence of triggers that generates the same event, S, on table T, often referred to as looping. It is important to ensure that recursive cascading does not produce undesired effects.

Definition (*Termination*). In the context of triggers, the concept of termination ensures that, regardless of the initial state and any sequence of modifications, there is always a definitive final state, thereby preventing the possibility of infinite activation cycles.

The simplest check exploits the triggering graph, that consists of a node i for each trigger T_i , and an arc from a node i to a node j if the execution of trigger T_i 's action may activate trigger T_j . The graph is created through a straightforward syntactic analysis. If it is acyclic, the system will definitely terminate. However, in the presence of cycles, triggers may or may not terminate. It's important to emphasize that acyclicity is adequate for ensuring termination but not a mandatory requirement.

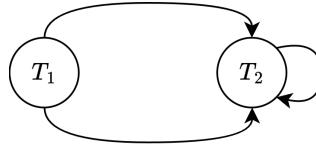
Example:

Consider the following triggers:

```
-- T1
CREATE TRIGGER AdjustContributions
AFTER UPDATE OF Salary ON Employee
REFERENCING NEW TABLE AS NewEmp
FOR EACH STATEMENT
UPDATE Employee
SET Contribution = Salary * 0.8
WHERE RegNum IN (SELECT RegNum FROM NewEmp)
-- T2
CREATE TRIGGER CheckOverallBudgetThreshold
AFTER UPDATE ON Employee
WHEN (SELECT sum(Salary+Contribution) FROM Employee) > 50000
UPDATE Employee
```

```
SET Salary = 0.9 * Salary;
```

The triggering graph associated with these triggers is illustrated below:



In this triggering graph, there are two cycles, but it's important to note that the system still terminates.

4.4 Triggers application

Triggers introduce robust data management capabilities in a seamless and reusable fashion. This system empowers databases to incorporate business and management rules that would otherwise be scattered across various applications. Nevertheless, comprehending the interplay between triggers can be intricate, as certain DBMS providers leverage triggers to execute internal functions.

View materialization A view is a virtual table defined through a query stored in the database catalog and subsequently utilized in queries as if it were a conventional table. When a view is referenced in a **SELECT** query, the query processor revises the query by employing the view definition, ensuring that the executed query solely involves the base tables linked to the view. If the queries involving a view significantly outnumber the updates to the base tables that alter the view's contents, view materialization can be considered. Certain systems offer the **CREATE MATERIALIZED VIEW** command, which enables the DBMS to automatically materialize the view. Alternatively, materialization can be implemented through the use of triggers.

Design principles The design principles encompass the following guidelines:

1. Employ triggers to ensure that specific operations trigger related actions.
2. Avoid defining triggers that replicate functionality already inherent in the DBMS.
3. Keep trigger code concise. If your trigger's logic extends beyond 60 lines of code, consider placing the majority of the code within a stored procedure and invoke the procedure from the trigger.
4. Utilize triggers exclusively for centralized, global operations intended to be executed for the triggering statement, regardless of the issuing user or database application.
5. Minimize the use of recursive triggers unless absolutely necessary, as triggers may inadvertently trigger one another until the DBMS exhausts its memory.
6. Exercise caution when implementing triggers, as they are executed for every user each time the relevant trigger event occurs.

Physical databases

5.1 Introduction

The structure of a physical database is organized as illustrated below:

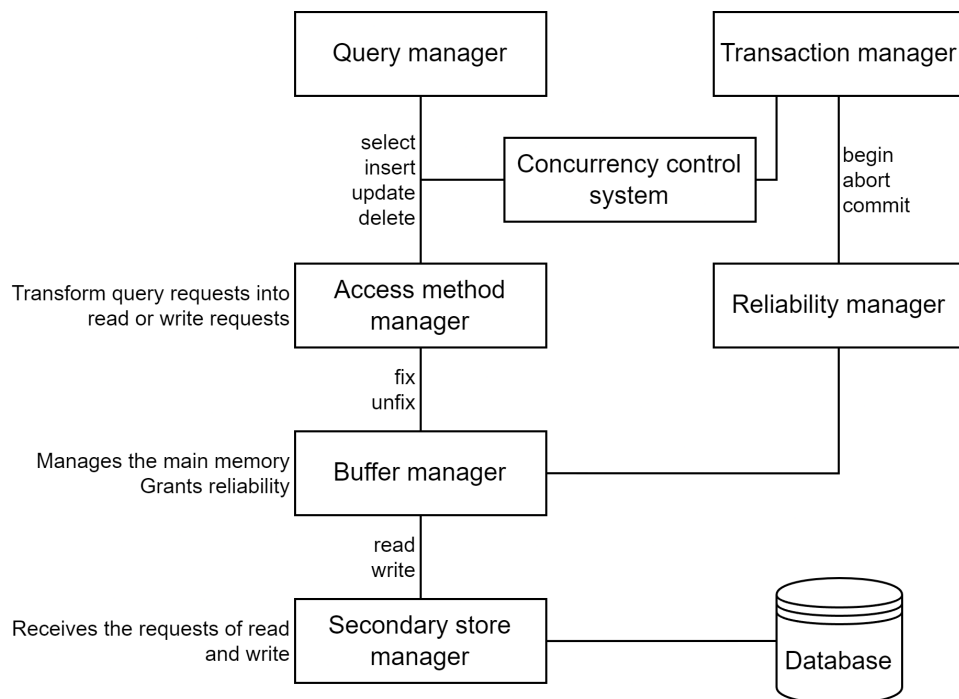


Figure 5.1: Architecture of a DBMS

Data are primarily stored in files on secondary memory due to considerations of size and persistence. Data in secondary memory becomes usable only when loaded into main memory through an I/O operation. The storage units are referred to as:

- *Block*, when in secondary memory, with a fixed length of a few kilobytes.
- *Page*, when in primary memory, assuming a size equivalent to that of blocks.

The time required for reading a block from a disk depends on the storage technology: Hard Disk Drive or Solid State Drive.

Hard disk drive Hard Disk Drives involve multiple disks stacked and rotating at a constant angular speed. A head stack, mounted on an arm, moves radially to reach tracks at various distances from the rotation axis, called spindle. A specific sector is accessed by waiting for it to pass under one of the heads. Multiple blocks can be accessed simultaneously (equal to the number of heads/disks).

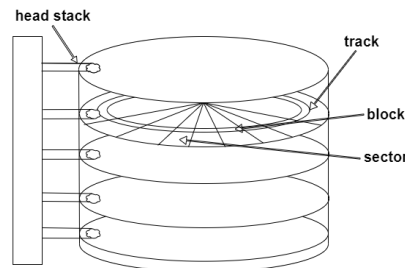


Figure 5.2: Structure of a Hard Disk Drive

Access time The secondary memory access time comprises:

- *Seek time* (8-12ms): head positioning on the correct track.
- *Latency time* (2-8ms): disc rotation on the correct sector.
- *Transfer time* (~ 1 ms): data transfer.

The cost of accessing secondary memory is four orders of magnitude higher than that of main memory. In many applications, the cost depends solely on the number of accesses to secondary memory. The cost of a query is closely related to the amount of data read (moved) from secondary memory.

File System The File System, the operating system layer managing secondary memory, is utilized minimally by the DBMS. DBMS directly manage file organization, both in data distribution within blocks and internal block structures. Additionally, a DBMS may control the physical allocation of blocks on the disk to optimize access time or enhance reliability.

5.2 Physical access structures

Definition (*Access methods*). The access methods are software modules that provide data access and manipulation primitives for each physical data structure.

Each DBMS has a distinctive and limited set of access methods. These methods employ specific data structures to organize data, with each table being stored in precisely one primary physical data structure, and potentially having one or more optional secondary access structures.

Classification These structures are categorized into:

- *Primary structure*: contains all the tuples of a table. Its primary purpose is to store the table content.

- *Secondary structures*: used to index primary structures, they only contain the values of certain fields interleaved with pointers to the blocks of the primary structure. Their primary function is to accelerate the search for specific tuples based on designated search criteria.

Three main types of data access structures exist: sequential, hash-based, and tree-based.

	Primary	Secondary
Sequential structures	Typical	Not used
Hash-based structures	In some DBMS	Frequent
Tree-based structures	Rare	Typical

Blocks and tuples Blocks serve as the physical components of files, while tuples are the logical components of tables. The block size is usually fixed and depends on the file system and disk formatting, whereas the tuple size (also known as a record) is variable within a file and contingent on the database design. For sequential and hash-based methods, a block is subdivided into the following components:

- Block header and trailer containing control information utilized by the File System.
- Page header and trailer with control information related to the access method.
- Page dictionary comprising pointers to each elementary item of useful data within the page.
- A useful part containing the data.
- Typically, page dictionaries and useful data expand as a stack in opposite directions.
- A checksum to verify the integrity of the block.

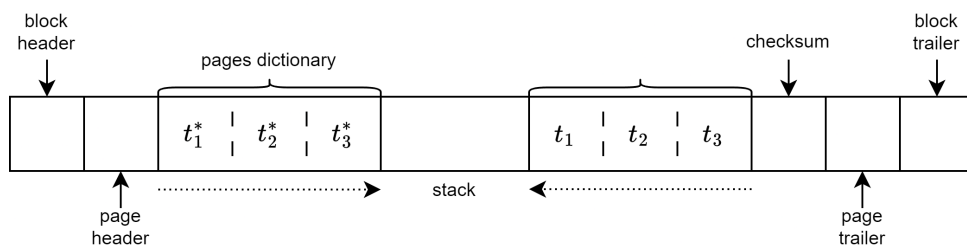


Figure 5.3: Structure of a block

The block factor (B) denotes the number of tuples within a block. To assess query costs, the following information is required:

- SR: the average size of a record (or tuple).
- SB: the average size of a block.

The block factor is defined as:

$$B = \left\lfloor \frac{SB}{SR} \right\rfloor$$

The remaining space can either be used, if the records are spanned between blocks, or not used if the records are unspanned.

Buffer manager Operations are executed in main memory and impact pages. In our cost model we assume pages of equal size and organization as blocks. The buffer operations encompass:

- *Insertion* and update of a tuple: may require a reorganization of the page or usage of a new page, also update to a field may require reorganization.
- *Deletion* of a tuple: typically accomplished by marking the tuple as invalid, with subsequent reorganization of the page performed asynchronously.
- *Access* to a field of a particular tuple: identified according to an offset with respect to the beginning of the tuple and the length of the field itself.

5.2.1 Sequential structures

In the context of sequential structures, we encounter an ordered arrangement of tuples in secondary memory, allowing for efficient access to the next element. Blocks can be either contiguous on disk or sparse. Two main possibilities arise:

- *Entry-sequenced organization*: sequence of tuples dictated by their order of entry.
- *Sequentially-ordered organization*: tuples ordered according to the value of a key.

Entry-sequenced organization Entry-sequenced organization, also known as a heap, represents the simplest and most common type of sequential organization. This organization proves to be efficient for:

- Insertion: this operation does not require shifting.
- Space occupancy: it utilizes all available blocks for data and all the space within a block.
- Sequential reading and writing: especially advantageous if the blocks are contiguous.
- Query like `SELECT * FROM table`.

However, it is inefficient for:

- Searching specific data units: this may require scanning the entire structure, a challenge mitigated by using indexes.
- Updates that increase tuple size: this may necessitate shifting and writing to another block. Deleting old versions of tuples and inserting new ones can address this issue.
- Query like `SELECT * FROM table WHERE <condition>`.

Sequentially-ordered sequential structure In this organization, tuples are sorted based on the value of a key field. It is efficient for:

- Range queries: retrieving tuples with the key in a specified interval.
- `ORDER BY` and `GROUP BY` queries: exploiting the key for sorting and grouping.

However, it is inefficient for reordering tuples within a block, especially challenging if space is limited. To address the global reordering issue, various techniques can be employed:

- Differential files and periodic merging.
- Local reordering operation within a block.
- Creation of an overflow file that contains tuples that do not fit in the current block.

Summary In real-world applications, the entry-sequenced organization is the most common solution only if paired with secondary access structures.

	Entry sequenced	Sequentially ordered
INSERT	Efficient	Not efficient
UPDATE	Efficient	Not efficient
DELETE	Invalid	Invalid
Tuple size	Fixed or variable	Fixed or variable
SELECT * FROM T WHERE <condition>	Not efficient	Efficient

5.2.2 Hash-based structures

Hash-based access structures provide efficient associative access to data based on the value of a key. The structure comprises N_B buckets ($N_B \ll$ number of data items), with each bucket being a unit of storage, typically equivalent to the size of one block. These buckets are often stored adjacently in the file.

A hash function maps the key field to a value between 0 and $N_B - 1$, interpreted as the index of a bucket in the hash structure (hash table). This organization proves efficient for:

- Tables with small size and (almost) static content.
- Point queries: queries with equality predicates on the key.

However, it is inefficient for:

- Range queries and full table queries.
- Tables with highly dynamic content.

Hash function The implementation of the hash function consists of two parts:

1. *Folding*: transforms key values into positive integer values, uniformly distributed over a large range.
2. *Hashing*: transforms the positive number into a value between 0 and $N_B - 1$ to identify the appropriate bucket for the tuple.

Collisions Collisions occur when two keys (tuples) are associated with the same bucket. When the maximum number of tuples per block is exceeded, collision resolution techniques are applied:

- *Closed hashing (open addressing)*: tries to find a slot in another bucket in the hash table. Linear probing is a simple technique, but not commonly used in databases.
- *Open hashing (separate chaining)*: allocates a new bucket for the same hash result, linked to the previous one.

Finding a tuple corresponding to a given key typically requires one access, although occasionally more.

Overflow chain The cost of accessing the tuple can be estimated by considering the average length of the overflow chain, which is influenced by:

- The load factor, that is the ratio of the occupied slots over the available slots:

$$\frac{T}{B \cdot N_B}$$

- The block factor B .

Here, T is the number of tuples, N_B is the number of buckets, and B is the number of tuples within a block. The average of accesses to the overflow list is summarized in the following table:

		Block factor				
		1	2	3	4	5
Load factor	50%	0.5	0.177	0.087	0.031	0.005
	60%	0.75	0.293	0.158	0.066	0.015
	70%	1.167	0.494	0.286	0.136	0.042
	80%	2.0	0.903	0.554	0.289	0.110
	90%	4.495	2.146	1.377	0.777	0.345

Example:

For an operation with a block factor of three and a load factor of seventy percent, the table indicates an average of 0.286 accesses to the overflow list. Consequently, the number of I/O operations is approximately 1.3.

Indexing Hash-based structures can be employed for secondary indexes, shaped and managed like a hash-based primary structure. Instead of tuples, the buckets only contain key values and pointers. Indexing without an overflow chain incurs a minimum cost of 2 I/O operations.

Summary In a large database, searching all index values to reach the desired data is inefficient. Hash-based structures exhibit good performance for equality predicates on the key field but prove inefficient for access based on interval predicates or the value of non-search-key attributes.

5.2.3 Indexes

Indexes are data structures designed to efficiently retrieve tuples based on specific criteria, known as a search key. Index entries are sorted with respect to the search key and typically contain records in the form

⟨search key, pointer to block⟩

Primary and search keys It's crucial to distinguish between the primary key, used for uniquely identifying a tuple, and the search key, employed for retrieving tuples based on specific criteria.

	Primary key	Search key
Access path defined	×	✓
Constraint defined	✓	×
Unique	✓	×
Implementation by index	✓	×

Index classification Indexes are smaller than primary data structures and can be loaded into a file in the main memory. They efficiently support point queries, range queries, and sorted scans. However, adding indexes to tables requires the DBMS to update each index after an insert, update, or delete operation, which can be costly and may slow down operations.

Definition (*Dense index*). An index is considered dense when it has an index entry for each search-key value in the file (one index per tuple).

This type of index provides excellent performance since a search on the index is sufficient to access the correct tuple. Dense indexing is applicable to entry-sequenced primary structures.

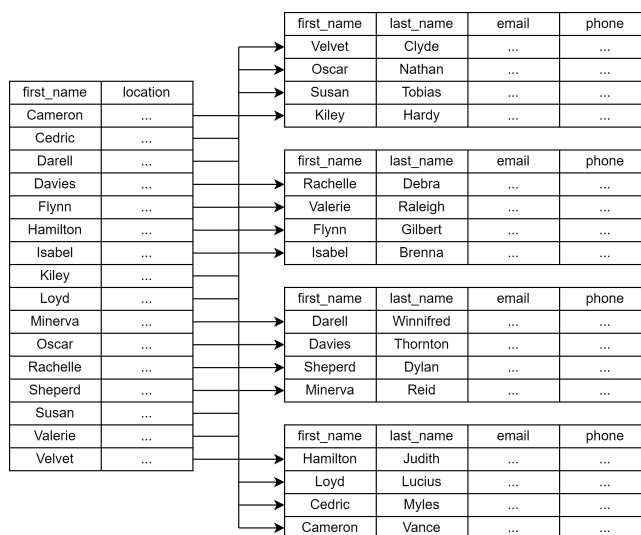


Figure 5.4: Example of dense indexing

Definition (*Sparse index*). An index is termed sparse when it has index entries only for some search-key values in the file (one index per block).

While this requires less space, the search process is slower as it involves scanning an entire block to find the tuple. Sparse indexing necessitates ordered data structures and primary indexing, providing a favorable trade-off with one index entry for each block in the file.

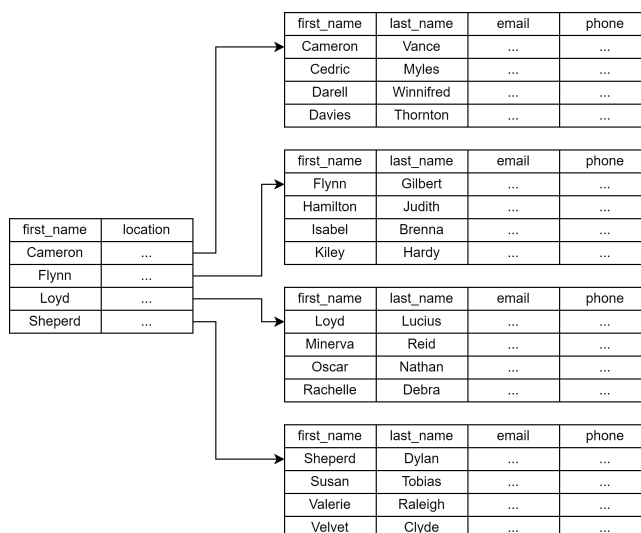


Figure 5.5: Example of sparse indexing

Primary index A primary index is defined on sequentially ordered structures. The search key (SK) is unique and coincides with the attribute according to which the structure is ordered (search key = ordering key). Only one primary index per table can be defined, typically on the primary key, but not necessarily. The primary index can be dense or sparse.

Clustering index A clustering index is a generalization of the primary index where the ordering key may not be unique. The pointer of key X refers to the block containing the first tuple of key X . It can be sparse or dense, but typically it's sparse.

Secondary index A secondary index has a search key specifying an order different from the sequential order of the file. Multiple secondary indexes can be defined for the same table, each on different search keys. It is necessarily dense because tuples with contiguous values of the key can be distributed in different blocks.

Summary The indexes are smaller than primary data structures, so they can be loaded into primary memory. They efficiently support point queries, range queries, and sorted scans. However, they are less efficient than hash structures for point queries. Adding indexes to tables requires the DBMS to update each index after an insert, update, or delete operation, incurring potential costs and slowing down data-changing operations.

Indexes in SQL To create an SQL index, every table should have:

- A suitable primary storage, possibly sequentially ordered.
- Several secondary indexes, both unique and non-unique, on the attributes most used for selections and joins. Secondary structures are progressively added, checking that they are used by the system.

Guidelines for choosing indexes:

1. Do not index small tables.
2. Index the primary key of a table only if it is not a key of the primary file organization.
3. Add a secondary index to any column heavily used as a secondary key.
4. Add secondary index structures on columns involved in **SELECT** or **JOIN** criteria, **ORDER BY**, **GROUP BY**, and other operations involving sorting.
5. Avoid indexing a column or table that is frequently updated.
6. Avoid indexing a column if the query will retrieve a significant number of rows.
7. Avoid indexing columns that consists of long strings.

The SQL commands used to create and remove the indexes are as follows:

```
CREATE [UNIQUE] INDEX <index_name> ON <table_name>[(<column_name>, ...)]  
DROP INDEX <index_name>
```

5.2.4 Tree-based structures

Frequently used in relational DBMSs (DBMS) for secondary index structures, balanced trees support associative access based on the value of a key search field. These trees are specifically balanced in that the lengths of the paths from the root node to the leaf nodes are all equal, contributing to enhanced performance. There are two main types of balanced trees used in this context:

- *B trees*: key values are stored in both internal and leaf nodes.
- *B+ trees*: all key values are stored only in leaf nodes.

B+ trees The B+ tree is an evolution from the B-tree, with each node stored in a block. Key values are exclusively stored in the leaf nodes, making B+ trees more efficient than B-trees, especially when a majority of the nodes are leaves. The fan-out of B+ trees depends on the size of the block, the key, and the pointer.

Internal node's structure Each node contains F keys, sorted lexicographically, and $F + 1$ pointers to child nodes. Each key K_j , $1 \leq j \leq F$ is followed by a pointer P_j , with K_1 preceded by a pointer P_0 . Each pointer addresses a subtree, with P_0 pointing to the subtree containing keys less than K_1 , P_F pointing to the subtree containing keys greater or equal to K_F , and intermediate pointers addressing subtrees containing keys K within the interval $K_j \leq K < K_{j+1}$. The value of F depends on the size of the page and the amount of space occupied by the key values and the pointers.

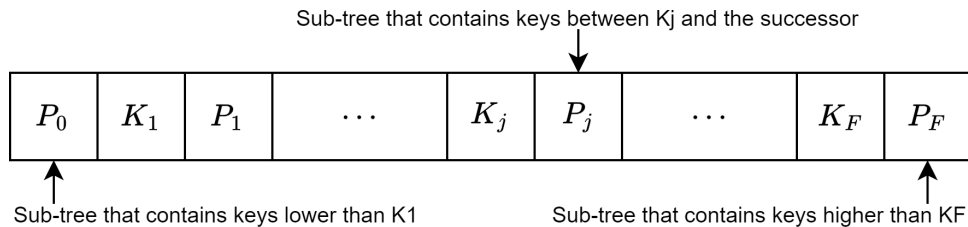


Figure 5.6: Structure of an internal node

Leaf node's structure The leaf node's structure resembles that of an internal node but contains only pointers to data tuples or the data tuples themselves. Leaf nodes can be structured in two ways:

1. The leaf node contains the entire tuple, making it key-sequenced.
2. The leaf node contains pointers to blocks of the database that contain tuples with the same key value, making it indirect.

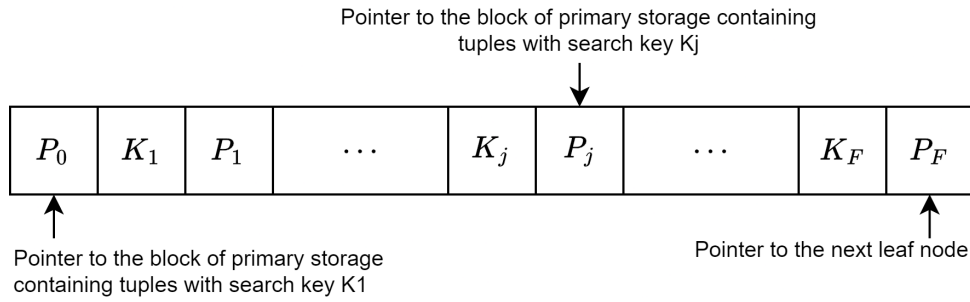


Figure 5.7: Structure of a leaf node

Search Mechanism The search mechanism involves following pointers starting from the root. At each intermediate node:

- If $V < K_1$, follow the pointer P_0 .
- If $V \geq K_F$, follow the pointer P_F .
- Otherwise, follow the pointer P_j such that $K_j \leq V < K_{j+1}$.

The search continues until a leaf node is found. In a key-sequenced leaf node, the search is complete as the tuple is found. In an indirect leaf node, it's necessary to access the memory block pointed to by the pointer P_j , $0 \leq j \leq F$.

Differences B+ trees have linked leaf nodes through a chain of pointers, ordered by key values. This chain enables efficient execution of range queries, allowing sequential scanning of leaves for values within a given range. This ordered scan is not possible in B trees, where accessing tuples in a specific range requires a search for each tuple. B trees use two pointers for each value k_i in intermediate nodes, saving space in the index pages and terminating the search when a key value is found without continuing the search in subtrees.

5.3 Query optimization

The DBMS's architecture includes a crucial component known as the optimizer. This optimizer plays a key role in processing SQL queries: it takes a query, conducts lexical, syntactic, and semantic analyses, and then generates an internal program using data access methods. Since there can be multiple ways to execute the same query, the optimizer is responsible for selecting the most efficient approach. The optimization process involves several steps:

1. Lexical, syntactic and semantic analysis of the query.
2. Translation into an internal representation (similar to algebraic expressions).
3. Algebraic optimization.
4. Cost-based optimization.
5. Code generation.

5.3.1 Relation profiles

Every commercial DBMS possesses quantitative information about the relations in the database, known as relation profiles. These profiles include:

- The cardinality (number of tuples) $\text{card}(T)$ of each table T .
- The dimension in bytes $\text{size}(T)$ of each tuple in table T .
- The dimension in bytes $\text{size}(A_j, T)$ of each attribute A_j in table T .
- The number of distinct values $\text{val}(A_j, T)$ of each attribute A_j in table T .
- The minimum and maximum values $\text{min}(A_j, T)$ and $\text{max}(A_j, T)$ of each attribute A_j in table T .

Relation profiles are derived from the data stored in tables and are periodically updated using system primitives. Cost-based optimization relies on the approximate values obtained from these relation profiles. The selectivity of a predicate, representing the probability that any row satisfies a given predicate, is a crucial metric. If $\text{val}(A) = N$ and the values are uniformly distributed, the selectivity of the predicate $A = v$ is $\frac{1}{N}$. If no data on distribution is available, then the distribution is always assumed uniform.

Example:

Consider a memory with block size of 8 KB (8192 bytes). Consider the following schemas:

1. STUDENT (ID, Name, LastName, Email, City, Birthdate, Sex)
2. EXAM (SID, CourseID, Date, Grade)

In the first table we have 150000 tuples. Every tuple has a size of 95 bytes. Each block contains:

$$\frac{8192}{95} \approx 87 \text{ tuples}$$

As a result the occupied blocks are:

$$\frac{150000}{87} \approx 1700 \text{ blocks}$$

In the second table we have 1,8 million tuples. Every tuple has a size of 24 bytes. Each block contains:

$$\frac{8192}{24} \approx 340 \text{ tuples}$$

As a result the occupied blocks are:

$$\frac{1800000}{340} \approx 5300 \text{ blocks}$$

5.3.2 Internal representation of queries

The optimizer creates a representation for a query that accounts for the physical structure used to implement tables and any present indexes. This internal representation utilizes a tree structure, where leaves correspond to physical data structures and intermediate nodes represent data access operations. Operations on intermediate nodes include sequential scans, orderings, indexed accesses, and joins.

Scan operation A scan operation sequentially accesses all tuples of a table, performing algebraic or extra algebraic operations such as projection, selection, ordering, insertions, deletions, and modifications.

Indexed access operation As already seen, indexes are structures of the database that allow to access the tuples of a table in a more efficient way than a sequential scan. The DBMS chooses the corresponding index when a query involves a single supported predicate. For queries with conjunctions or disjunctions of predicates, the DBMS optimizes the use of indexes.

Join The JOIN operation is acknowledged as the most resource-intensive task for a DBMS due to the potential for a substantial increase in the number of tuples in the result. To address this challenge, the optimizer employs one of three techniques:

1. *Nested-loop*, based on scanning:

- A scan is executed on a table (referred to as the external table), and for each tuple, a scan is performed on the other table (referred to as the internal table).
- The matching process is efficient when there is an index on the join attribute of the internal table.

2. *Merge-Scan*, based on ordering:

- This technique necessitates that both tables are sorted based on the join attribute.
- Two synchronized scans traverse the tuples of each table in parallel, and the matching is carried out.

3. *Hash-join*, based on hashing

- This technique requires that both tables are hashed using the join attribute.
- The tuples of the internal table are hashed and stored in a hash table, while the tuples of the external table are scanned, and the matching is performed.

Equality The expense associated with an equality lookup is contingent upon the type of structure that represents the table:

- *Sequential structures with no index:*

- Equality lookups are not supported, resulting in a cost equivalent to a full scan.
- Sequentially ordered structures might incur a reduced cost.

- *Hash or Tree structures:*

- Equality lookups are supported if A is the search key attribute of the structure.
- The cost depends on the storage type (primary or secondary) and the search key type (unique or not).

Range The cost associated with a range lookup is contingent upon the type of structure representing the table:

- *Sequential structures* (primary):
 - Range lookups are not supported, resulting in a cost equivalent to a full scan.
 - Sequentially ordered structures may incur a reduced cost.
- *Hash structures* (primary and secondary):
 - Range lookups are not supported, leading to a cost equivalent to a full scan.
- *Tree structures* (primary and secondary):
 - Range lookups are supported if A serves as the search key attribute of the structure.
 - The cost depends on the storage type (primary or secondary) and the search key type (unique or non-unique).
- *B+ tree structures* (primary and secondary):
 - Range lookups are supported if A is the search key attribute of the structure.

Conjunction In the presence of indexes, the DBMS opts for the most selectively supported predicate for data access. Subsequently, the remaining predicates are assessed later in main memory through a sequential scan.

Disjunction If any of the predicates lack index support, a sequential scan is executed. However, if indexes are available for all predicates, the DBMS utilizes them to assess each predicate individually. The results are then merged, and duplicates are subsequently removed.

Sort Various methods facilitate the optimal sorting of a table's tuples. However, a significant challenge arises from the necessity for the DBMS to load the entire data into the buffer, a task that may exceed the available memory capacity. Sorting can occur either in main memory (utilizing ad-hoc algorithms like quick sort or merge sort) or on disk (employing external sorting algorithms). The latter approach is chosen when the data surpasses the main memory's capacity, following this procedure:

1. Split the data into chunks, each of a size equal to the main memory.
2. Load a chunk into the main memory.
3. Sort the data in the main memory.
4. Store the sorted data back on disk.
5. Merge sorted chunks using at least three pages:
 - Two for progressively loading data from two sorted chunks.
 - One as an output buffer to store the sorted data.
6. Save the result on disk.
7. Repeat from step two until all chunks are merged.

5.3.3 Cost-based Optimization

Cost-based optimization poses a challenge in decision-making within an optimization problem, involving:

- Determining which data access operations to execute.
- Deciding in which order to perform the operations.
- Choosing which option to select if there are multiple choices for a given operation.
- Establishing how to sort the data if sorting is required in the query.

To address this challenge, the DBMS leverages data profiles and approximate cost formulas. A decision tree is constructed, where:

- Each internal node represents a decision point (a choice between two or more options).
- Each leaf node represents a specific plan (a sequence of operations).

Assigning a cost to each plan allows the identification of the optimal one using operations research techniques like branch and bound. Optimizers must efficiently handle such problems within a reasonable timeframe.

5.3.4 Approaches to Query Evaluation

The query is evaluated by the DBMS according to two techniques.

Compile and store The query undergoes compilation and is stored in the DBMS for later execution. The internal code is retained in the DBMS along with indications of dependencies on specific versions of the catalog used during compilation. Upon relevant changes in the catalog, the compilation of the query is invalidated, leading to the recompilation of the query.

Compile and go The query is compiled and executed immediately, without storing the compiled code. Although the code is not stored permanently, it may persist in the DBMS for a period to be potentially reused by subsequent executions.

Multidimensional indexing

6.1 Introduction

The demand for multidimensional data emerged in the 1970s, primarily driven by the need to manage Geographic Information Systems (GIS) and Computer-Aided Design (CAD). Two decades later, the inception of multimedia databases and the introduction of data mining techniques marked significant milestones. These advancements paved the way for the evolution of spatial DBMSs (DBMS). The application of this technique extends beyond spatial databases, finding utility in classical databases featuring multiple numerical attributes.

Applications The applications of multidimensional data are diverse, encompassing:

- *Geographic Information Systems (GIS)*: in GIS applications, the data includes both points (such as places and cities) and objects with spatial extensions (e.g., regions, streets, and rivers).
- *Multimedia databases*: multimedia content is represented by numerical characteristics known as features. In this context, two pieces of content are considered equal if their respective features are similar.
- *Relational databases*: this technique is employed to identify the optimal combination of multiple attributes based on user or system criteria.

Query classification Multidimensional databases support various query types, including:

- *Point/lookup query*: examines the contents of a single point in space, useful for detecting duplicates within a specific location.
- *Window query*: combines two different attributes to inspect all elements within a triangular region.
- *Range query*: starting from a point and a radius, it searches for all elements within a circular area with the specified center and radius.
- *K-nearest neighbor*: given a point in space, this query type identifies the best k nearest elements based on a defined criterion.

- *Specific query types for GIS systems*: examples include queries related to streets crossing a railroad or buildings in proximity to a river.

Using B+ trees Consider a window query involving two attributes, A and B . Assume that each query interval selects 10% of the total number of points, resulting in an expectation of retrieving 1% of the data when the attributes are independent. When employing B+ trees, two potential solutions arise:

1. Implement a single B+ tree for both attributes by sorting first on A and then on B . However, this approach leads to reading numerous leaves that do not contain any points in the result set.
2. Use two separate B+ trees for each attribute. In this scenario, 10% of leaves are read on both indices, yet the intersection only contains 1% of the data.

In both cases, excessive effort is wasted due to the storage of spatially close points in distant leaves. In the first case, this is attributed to the linearization of attributes, while in the second case, it is a consequence of disregarding other attributes. As additional attributes are introduced, performance deterioration persists.

To address these challenges, multidimensional indices can be employed. These specialized indices aim to maintain the spatial proximity of records (Local Order Preservation) by grouping objects in pages, ensuring that each page contains objects that are close in the d -dimensional space. This task is non-trivial, given that a global order is not defined in d dimensions.

6.2 Data partitioning

The spatial indices structure can be derived either from one-dimensional structures (such as a grid file) or from ad-hoc structures (like an R-tree), resulting in a myriad of potential data structures.

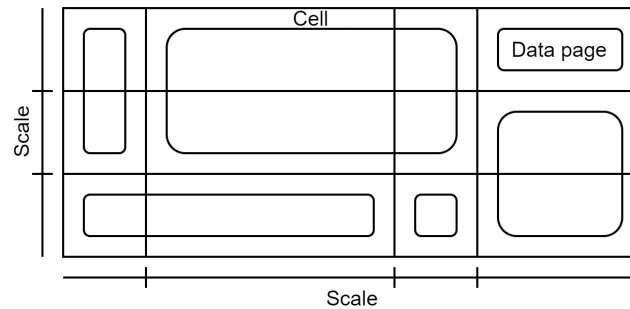
Classification Spatial indices can be classified based on:

- *Type of objects*: points, lines, and regions.
- *Type of subdivision*:
 - Space partitioning: splits are executed according to global considerations, which is beneficial for uniform distributions and easier to implement.
 - Data partitioning: splits are performed based on local considerations, akin to B+-trees. This method is suitable for arbitrary distributions but is more complex to implement.
- *Type of organization*: either tree-based or hash-based.

The prevalent approach involves organizing space into regions, referred to as cells, where each cell corresponds to a data page (although this correspondence is not always one-to-one).

6.2.1 Grid files

The Grid file arranges the spatial domain into a grid, with each dimension divided into intervals of arbitrary sizes. This entails the use of d scales (monodimensional arrays) that hold values serving as separators for each dimension. Each grid cell is linked to a data page on disk, allowing for the possibility of two or more cells sharing the same data page.

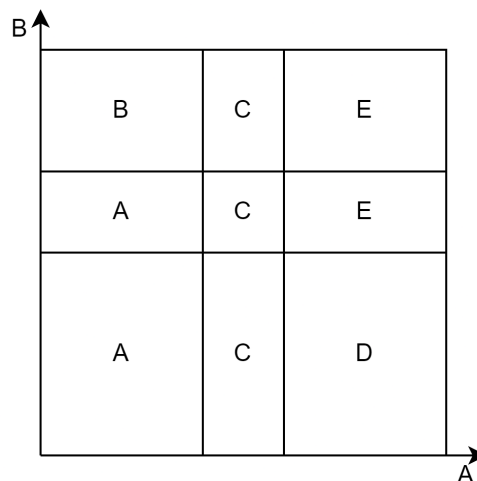


When a data page reaches its capacity and overflows, it undergoes a split. After a split, the directory structure changes by applying one of the following actions:

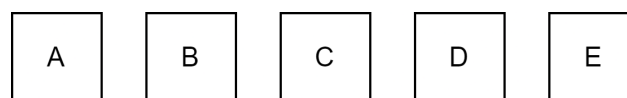
- If the block was referenced by two or more cells, we simply update the pointers.
- Otherwise, if the block was referenced by a single cell, we add a separator to the directory.

Example:

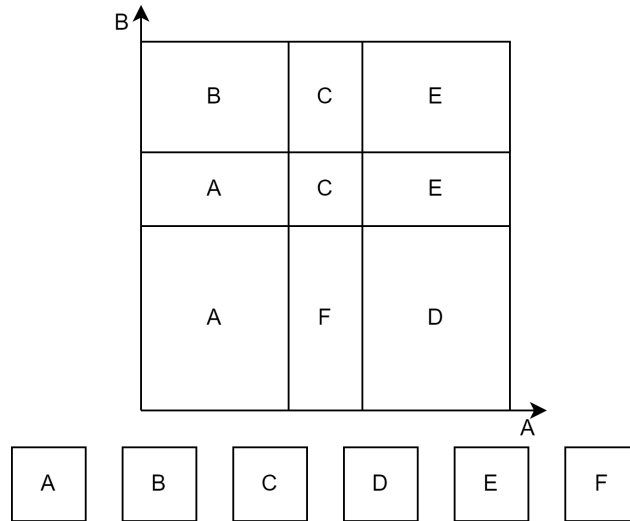
Examine the given directory:



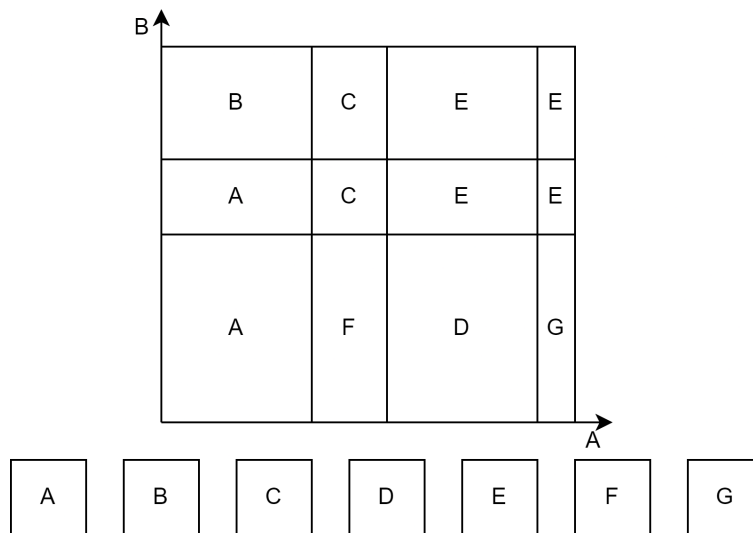
Associated with the depicted data pages below:



Now, consider the scenario where C overflows and undergoes a split into C and F . In this case, it is sufficient to update the pointer of the cell. The resulting directory and data pages are as follows:



Next, consider the case where D overflows and is split into D and G . In this scenario, we need to augment the directory using an additional separator for coordinate A . The resulting directory and data pages become:



Summary For non-uniform distributions, the storage of n points may demand a number of cells that increases proportionally to $O(n^d)$. In general, this approach exhibits poor performance with correlated data. Conversely, the structured nature of space partitioning significantly simplifies the resolution of window queries.

The primary challenge with this technique lies in the management of the directory. Typically, scales are stored in main memory. In quasi-static scenarios, the directory can be stored on disk as a multidimensional array. However, in dynamic cases, it becomes essential to paginate the directory, resulting in multi-level grid files.

6.2.2 R-tree

An R-tree is a well-balanced and paginated tree structure built upon the hierarchical nesting of overlapping regions. In this structure, every node represents a rectangular region, specifically the Minimum Bounding Box (MBB) encapsulating its children's regions. The storage utilization

for each node ranges from 100% down to a minimum value ($\leq 50\%$), a parameter set during design. The management mechanisms in an R-tree bear similarities to those found in a B+ tree. However, a key distinction lies in the fact that the insertion of an object and splits can be handled according to different policies.

Minimum Bounding Box The Minimum Bounding Box is the smallest rectangle containing all children regions, with sides parallel to the coordinate axes. It is determined by the product of d intervals and can be uniquely defined by the (2d) coordinates of any two opposite vertices, typically the lower-left and upper-right vertices. The computation of the MBB for m points in d dimensions can be performed in $O(md)$ time.

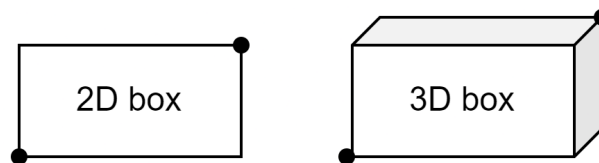
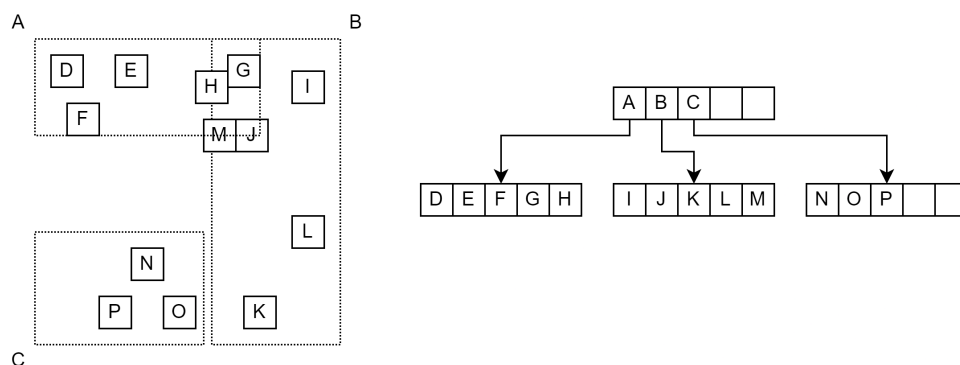


Figure 6.1: Graphical representation of a 2d and a 3d MBB

The differences between a B+ tree and an R-tree are outlined below:

B+ tree	R-tree
Balanced and paginated tree	Balanced and paginated tree
Data are stored in leaves	Data are stored in leaves
Leaves are kept sorted and linked	No data order exist
Data are organized into 1D intervals	Data are organized into d-dim intervals
This principle is recursively applied towards the root	This principle is recursively applied towards the root
Point search follows a single path from root to a single leaf	Point search could follow multiple paths from the root to multiple leaves

Organization The R-tree is constructed by recursively aggregating objects in a bottom-up fashion, utilizing the Minimum Bounding Box (MBB). Overlapping regions may be present within the MBBs. Each node, excluding the root, has a capacity for a maximum of $C_{\max}$ entries, but no fewer than $C_{\min} = 0.5 \cdot C_{\max}$. Visually, the structure can be represented as shown below:



Leaf nodes store entries in the form $\langle \text{key}, \text{RID} \rangle$, where the key holds the coordinates of the record referenced by RID. Additionally, the R-tree is capable of storing d -dimensional objects with spatial extensions, where $\text{key} = \text{MBB}(\text{object})$.

Internal nodes contain entries of the form $\langle \text{MBB}, \text{PID} \rangle$, where MBB represents the coordinates of the MBB containing all entries reachable from the node referenced by PID. In summary, each node contains entries in the format $\langle \text{key}, \text{pointer} \rangle$, where the key is a spatial value.

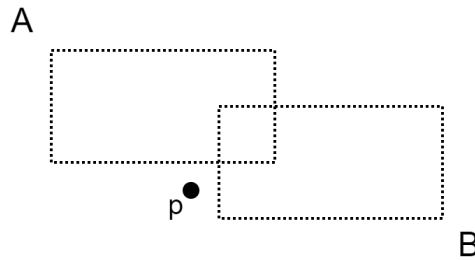
Insertion To **insert** a new object, the process begins at the root and proceeds down the tree incrementally, seeking an optimal location to accommodate the new object p . At each step, the objective is to identify the most appropriate child node for p . In general, a penalty value is computed for each node, and the one with the minimum penalty is selected.

If point p falls within the region of a node, the penalty is set to 0. Otherwise, the penalty is determined by the increase in volume (or area) of the Minimum Bounding Box (MBB). For leaf nodes, the R^* tree variant considers the increment of overlap with other regions. Both criteria are designed to enhance tree performance, with the following considerations:

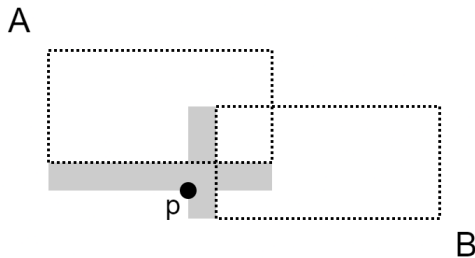
- *Large volume*: increases the likelihood of visiting the node during a query.
- *Large overlap*: raises the number of nodes visited during a query.

Example:

Consider the example of inserting the point p into an R-tree with nodes A and B :



In this scenario, the most suitable area for the point p is node B since it results in a lower area:



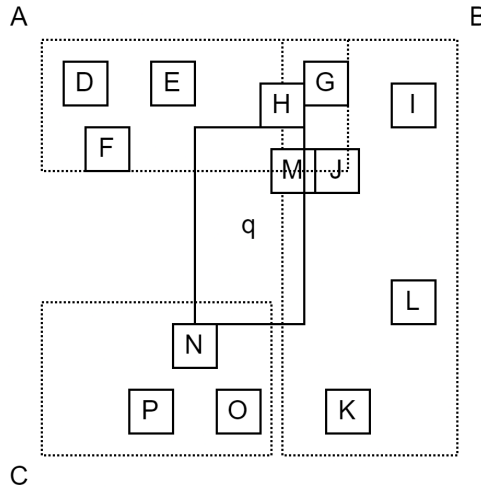
Splitting When the insertion of point p into a leaf node N results in surpassing the maximum capacity $C_{\max}$, an overflow occurs, necessitating the split of node N . For leaf nodes containing point entries, the goal is to divide the set of $C_{\max} + 1$ points into two subsets, each with a minimum of $C_{\min}$ and a maximum of $C_{\max}$ points. This task is recognized as an $\mathcal{NP}$ -hard problem, requiring the application of heuristics. Similar to B+ trees, splits propagate upward and may recursively induce splits at higher levels of the tree. The objective of the R-tree is

to minimize the sum of resulting volumes. In contrast, the R^* tree employs a more intricate criterion, considering the volume, overlap, and surface of the resulting regions. It should be noted that guaranteeing an overlap-free split is not always possible.

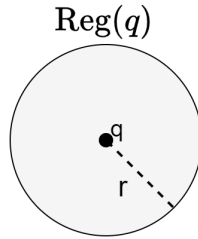
Search To search for objects, we need to retrieve all points contained within the product of d intervals. Such points are located in nodes whose Minimum Bounding Boxes (MBBs) overlap with the query region.

Example:

Consider a visual representation of a window query in the R-tree shown in the previous example:



Another type of query is the range query, which involves a query point q , a search radius r , and a distance function D . The query region is defined as $\text{Reg}(q) = \{p \mid p \in \mathbb{R}^d, D(p, q) \leq r\}$.



The algorithm for processing a range query is straightforward:

- Start from the root; for each node N , check if $\text{MBB}(N)$ intersects $\text{Reg}(q)$.
- On leaf nodes, for each point p , check if $D(p, q) \leq r$.

Search with distance functions For a node N , the minimum possible distance between the query point q and a point reachable from N is defined as:

$$D_{\text{MIN}}(q, N) = \inf_p \{D(q, p) \mid p \in \text{MBB}(N)\}$$

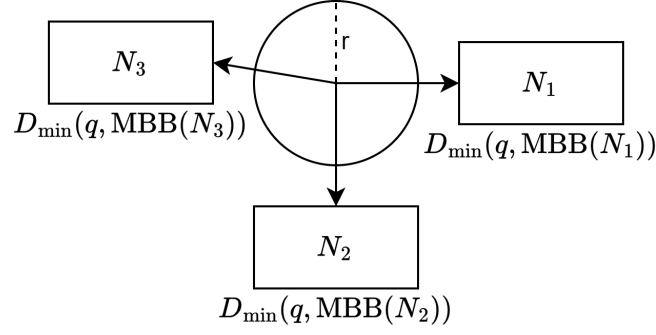


Figure 6.2: Graphical representation of a general minimum distance function

The fundamental characteristic of the distance function is expressed as:

$$\text{Reg}(q) \cap \text{MBB}(N) \neq \emptyset \leftrightarrow D_{\text{MIN}}(q, N) \leq r$$

The commonly used distance functions are L_p -norms, generally given by:

$$L_p(t, q) = \left(\sum_{i=1}^d |t_i - q_i|^p \right)^{\frac{1}{p}}$$

Notable instances of the L_p -norm include:

- *Euclidean distance*: $L_2(t, q) = \sqrt{\sum_{i=1}^d |t_i - q_i|^2}$
- *Manhattan distance*: $L_1(t, q) = \sum_{i=1}^d |t_i - q_i|$
- *Čebyšëv distance*: $L_\infty(t, q) = \max_i \{|t_i - q_i|\}$

These distance functions also have weighted versions:

- *Euclidean distance*: $L_2(t, q) = \sqrt{\sum_{i=1}^d w_i |t_i - q_i|^2}$
- *Manhattan distance*: $L_1(t, q) = \sum_{i=1}^d w_i |t_i - q_i|$
- *Čebyšëv distance*: $L_\infty(t, q) = \max_i \{w_i |t_i - q_i|\}$

For the i -th coordinate of a given Minimum Bounding Box (MBB), the offset of the query point $q = (q_1, \dots, q_d)$ with respect to $\text{MBB}(N)$ is defined as:

$$\delta_i = \begin{cases} q_i - h_i & \text{if } q_i \geq h_i \\ l_i - q_i & \text{if } l_i \geq q_i \\ 0 & \text{otherwise} \end{cases}$$

As a result, the distance D_{MIN} is computed as:

$$L_{p,\text{MIN}}(q, N; W) = \left(\sum_{i=1}^d w_i \delta_i^p \right)^{\frac{1}{p}}$$

Search with k-NN queries The `kNNOptimal` algorithm provides an I/O-optimal solution for k-NN queries with an R-tree. The algorithm is applicable to all index structures where the region of any child node is encompassed within the region of its parent.

The fundamental case involves setting $k = 1$. In this scenario, consider t_{NN} as the first nearest neighbor of q (i.e., 1-NN), with $r_{NN} = D(q, t_{NN})$ representing its distance from q . The distance r_{NN} becomes known only upon the termination of the algorithm.

Theorem 6.2.1. *Any correct algorithm for 1-NN queries must visit at least all nodes N such that $D_{\min}(q, N) < r_{NN}$.*

Proof. If an algorithm A stops by reporting a point t as the NN of q , and A does not read a node N such that $D_{\min}(q, N) < D(q, t)$, then N might lead to a point t' with $D(q, t') < D(q, t)$, contradicting the hypothesis that t is the NN of q . $\square$

The `kNNOptimal` algorithm employs a priority queue PQ, consisting of pairs $\langle \text{ptr}(N), D_{\min}(q, N) \rangle$. PQ is ordered based on increasing values of $D_{\min}(q, N)$. Thus, `dequeue` extracts the pair with the minimum $D_{\min}$, and `enqueue`($\langle \text{ptr}(N), D_{\min}(q, MBB(N)) \rangle$) performs an ordered insertion of the pair into the queue.

Property 6.2.1. If a point t with $D(q, t) = r$ is found, then all nodes N with $D_{\min}(q, N) \geq r$ can be excluded from the search.

The exclusion of nodes can be achieved through pruning. The `update(PQ)` function facilitates this operation. If node pruning is not desired, the nodes can be retained in the queue until the point where further improvement in the result is unattainable.

The algorithm takes a query point q and an index tree with the root node RN as input, producing outputs t_{NN} (the nearest neighbor of q) and $r_{NN} = D(q, t_{NN})$.

Algorithm 3 kNN-Optimal Algorithm

```

1: Initialize PQ with  $\langle \text{ptr}(RN), 0 \rangle$ 
2:  $r_{NN} := \infty$ 
3: while PQ  $\neq \emptyset$  do
4:    $\langle \text{ptr}(N), D_{\min}(q, N) \rangle := \text{dequeue}$ 
5:    $\text{read}(N)$ 
6:   if  $N$  is a leaf then
7:     for each point  $t \in N$  do
8:       if  $D(q, t) < r_{NN}$  then
9:          $t_{NN} := t$ 
10:         $r_{NN} := D(q, t)$ 
11:         $\text{update}(PQ)$ 
12:      end if
13:    end for
14:   else
15:     for each child node  $N_c \in N$  do
16:       if  $D_{\min}(q, N_c) < r_{NN}$  then
17:          $\text{enqueue}(\langle \text{ptr}(N_c), D_{\min}(q, N_c) \rangle)$ 
18:       end if
19:     end for
20:   end if
21: end while
22: return  $t_{NN}$  and  $r_{NN}$ 

```

The correctness of the **kNNOptimal** algorithm is evident. To establish its I/O-optimality, i.e., to demonstrate that it reads the minimum number of nodes, it suffices to prove the following theorem.

Theorem 6.2.2. *The **kNNOptimal** algorithm for 1-NN queries never reads a node N whose D_{MIN} is strictly larger than r_{NN} , i.e.:*

$$D_{MIN}(q, N) > r_{NN}$$

Proof. Node N is read only if, at some execution step, it becomes the first element in PQ. Let N_1 be the node containing t_{NN} , N_2 its parent node, N_3 the parent node of N_2 , and so on, up to $N_h = RN$, where h is the height of the tree. Now observe that, by definition of D_{MIN} , it is:

$$r_{NN} \geq D_{MIN}(q, N_1) \geq D_{MIN}(q, N_2) \geq \dots \geq D_{MIN}(q, N_h)$$

At each time step before we find t_{NN} , one (and only one) of the nodes $N_1, N_2, \dots, N_h$ is in the priority queue. It follows that N can never become the first element of PQ. $\square$

The algorithm easily extends to the case where $k > 1$ by incorporating:

- A data structure (Res), maintaining the k closest objects found so far along with their distances from q .
- A current search radius, rk -NN, representing the distance of the current k -th nearest neighbor of q (i.e., the k -th element of Res).

The remainder of the algorithm remains unchanged.

Indexing regions Leaves can store Minimum Bounding Boxes (MBBs) of regions instead of individual points. Suppose the objective is to identify all data regions o that are at a distance less than or equal to r from the query region, expressed as $L_2(q, o) \leq r$. This process involves two phases:

1. *Filter phase:* utilize the index to retrieve all MBBs such that $L_2(\text{MBB}(q), \text{MBB}(o)) \leq r$.
2. *Refine phase:* access the actual geometries of the objects and apply the test $L_2(q, o) \leq r$.

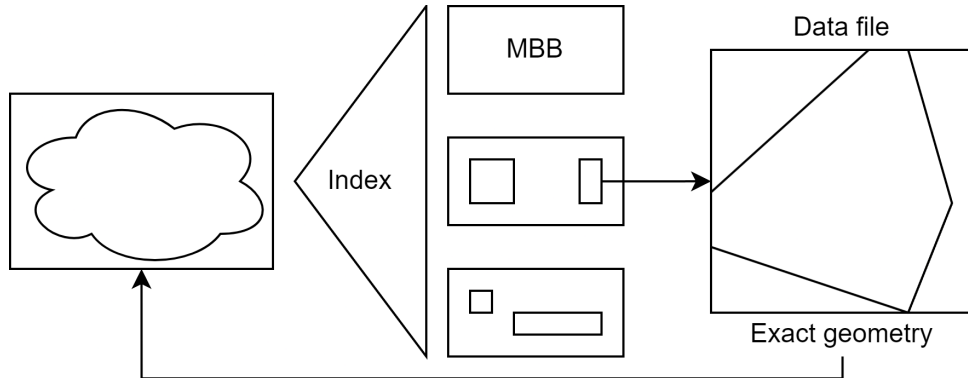


Figure 6.3: Representation of indexing regions procedure

6.3 Metric distances

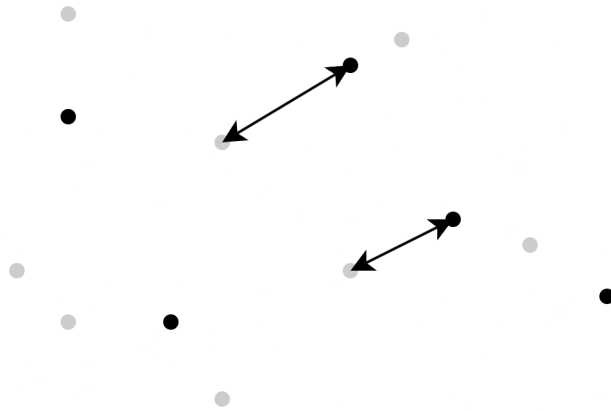
Vector spaces, even when equipped with (weighted) L_p -norms, lack the generality required to handle the diverse range of feature types and distance functions essential for multimedia (MM) data.

Metric spaces A metric space $M = (U, d)$ is defined as a pair, where:

- U represents a domain of values.
- d is a distance function that, for all $x, y, z \in U$, satisfies the metric axioms:
 - *Positivity*: $D(x, y) \geq 0, D(x, y) = 0 \leftrightarrow x = y$.
 - *Symmetry*: $D(x, y) = D(y, x)$.
 - *Triangle inequality*: $D(x, y) \leq D(x, z) + D(z, y)$.

All the distance functions previously discussed, as well as the (weighted) L_p -norms, adhere to the metric axioms. Metric indexes leverage these axioms to organize objects and utilize the triangle inequality for search space pruning.

Shape matching Consider two sets of points, S_1 and S_2 :



The Hausdorff distance is defined through the following steps:

1. For each gray point in S_1 , find the closest black point in S_2 . Let $h(S_1, S_2)$ be the maximum of these distances.
2. For each black point in S_2 , find the closest gray point in S_1 . Let $h(S_2, S_1)$ be the maximum of these distances.
3. Compute the Hausdorff distance as:

$$D_{\text{Haus}}(S_1, S_2) = \max\{h(S_1, S_2), h(S_2, S_1)\}$$

This technique is particularly useful for matching shapes.

Set similarity To calculate the difference and similarities between two sets, s_1 and s_2 , the function $D_{setdiff}(s_1, s_2)$ can be employed:

$$D_{setdiff}(s_1, s_2) = \frac{|s_1 - s_2| |s_2 - s_1|}{|s_1| + |s_2|}$$

Edit distance The edit distance is a commonly used measure for strings, defined as the minimum number of operations required (insertions, deletions, or substitutions) to transform one string s_1 into another string s_2 .

Example:

Consider the following pairs:

1. $\langle \text{ball, bull} \rangle$.
2. $\langle \text{balls, bell} \rangle$.
3. $\langle \text{rather, alter} \rangle$.

The corresponding edit distances are:

1. $D_{\text{edit}}(\text{ball, bull}) = 1$.
2. $D_{\text{edit}}(\text{balls, bell}) = 2$.
3. $D_{\text{edit}}(\text{rather, alter}) = 3$.

The edit distance is widely used in genomic databases to compare DNA sequences. Each DNA sequence is treated as a string over the 4-letter alphabet of bases.

Example:

Consider the pair $\langle \text{rather, alter} \rangle$. The edit distance computation involves two matrices. Initially, the cost matrix is defined as follows:

r	1	1	1	1	1	0
e	1	1	1	1	0	1
h	1	1	1	0	1	1
t	1	1	1	1	1	1
a	1	0	1	1	1	1
r	1	1	1	1	1	0
↑	0	1	1	1	1	1
→	a	l	t	e	r	

This matrix is utilized to incrementally construct the new matrix D_{edit} , with its elements recursively defined as:

$$D_{\text{edit}}^{(i,j)} = \text{cost}_{(i,j)} + \min \left\{ D_{\text{edit}}^{(i-1,j)}, D_{\text{edit}}^{(i,j-1)}, D_{\text{edit}}^{(i-1,j-1)} \right\}$$

In this case, the resulting table is as follows:

r	6	5	5	5	4	3
e	5	4	4	4	3	4
h	4	3	3	3	3	4
t	3	2	3	2	3	4
a	2	1	2	3	4	5
r	1	1	2	3	4	4
↑	0	1	2	3	4	5
→	a	l	t	e	r	

Fréchet distance The Fréchet distance, also known as the dog-keeper distance, serves as a superior measure of similarity between curves compared to the Hausdorff distance. This metric finds application in diverse fields such as speech recognition, signature and handwriting recognition, as well as the matching of time series and the analysis of moving objects. Consider an individual moving along a finite path while walking a dog on a leash, each following a distinct finite path. The Fréchet distance quantifies the length of the shortest leash required for both the person and the dog to traverse their respective paths entirely.

Metric indexing principles Given a metric dataset $P \subseteq U$, it can be partitioned into two subsets using one of the following principles:

1. *Ball decomposition*: start with a designated point v , referred to as the vantage point. Calculate the distances of all other points p in relation to v , denoted as $D(p, v)$. Then, define two subsets as follows:

$$P_1 = \{p \mid D(p, v) \leq r_v\}$$

$$P_2 = \{p \mid D(p, v) > r_v\}$$

For a given range query $p \mid D(p, q) \leq r$, if $D(q, v) > r_v + r$, it can be inferred that no point in P_1 belongs to the result. Similar reasoning applies to P_2 .

2. *Generalized hyperplane*: choose two points, v_1 and v_2 . Compute the distances of all other points p with respect to both v_1 and v_2 . Define two subsets as follows:

$$P_1 = \{p \mid D(p, v_1) \leq D(p, v_2)\}$$

$$P_2 = \{p \mid D(p, v_2) < D(p, v_1)\}$$

For a range query $\{p \mid D(p, q) \leq r\}$, if $D(q, v_1) - D(q, v_2) > 2r$, it implies that no point in P_1 contributes to the result.

6.3.1 M-tree

The M-tree has been the inaugural dynamic, paged, and balanced metric index, extending the principles of R-trees to arbitrary metric spaces. At a glance, the M-tree bears a resemblance to an R-tree. Nevertheless, it exclusively operates with distance values, disregarding coordinate values, and abstains from relying on any geometric reasoning.

Structure Objects are recursively aggregated in a bottom-up manner based on regions in a structure where regions are allowed to overlap. Each node, except the root, can house up to $C_{\max}$ entries, with a minimum requirement of $C_{\min} \leq 0.5 \cdot C_{\max}$

Each tree node N is associated with a region, denoted as $\text{Reg}(N)$, defined by:

$$\text{Reg}(N) = \{p \mid p \in U, D(p, v_N) \leq r_N\}$$

Here, v_N (referred to as the "center") is also known as a routing object, and r_N is termed the covering radius of the region. The set of indexed points p reachable from node N ensures that $D(p, v_N) \leq r_N$. This allows the application of the pruning principle: if $D(q, v_N) > r_N + r$, then prune node N .

In leaf nodes, an entry E in node N takes the form $E = \langle \text{ObjFeatures}, \text{distP}, \text{RID} \rangle$, where ObjFeatures represent the feature values of the indexed object, and distP is the distance between the object and its parent routing object (i.e., the routing object of node N).

For internal nodes, an entry E in node N is expressed as:

$$E = \langle \text{RoutingObjFeatures}, \text{CoveringRadius}, \text{distP}, \text{PID} \rangle$$

Here, $\text{RoutingObjFeatures}$ encompass the feature values of the routing object, CoveringRadius denotes the radius of the region, and distP represents the distance between the routing object and its parent routing object (undefined for entries in the root node).

Characteristics During query execution, pre-computed distances, denoted as distP , are leveraged to conserve computational resources. Let v_P represent the parent routing object of v_N . When examining the entry of v_N , the distance $D(q, v_P)$ between the query and its parent has already been computed. Consequently, we have knowledge of the distance $D(v_P, v_N)$. Applying the triangle inequality, we can deduce that:

$$D(q, v_N) \geq |D(q, v_P) - D(v_P, v_N)|$$

This allows us to prune node N without computing $D(q, v_N)$ if:

$$|D(q, v_P) - D(v_P, v_N)| > r_N + r$$

Operations The process of inserting a new object relies on a Penalty method. This method evaluates the increment in the covering radius required to incorporate the new object. When handling a split, there are various alternatives, including:

- **mM_RAD**: minimize the maximum of the two resulting radii.
- **M_LB_DIST**: choose the closest and farthest objects from v_N .

Experiments reveal that **mM_RAD** yields the best results.